

Index

Memory management pg 1 to 11
Fragmentation and Paging pg 11 to 32
Process Synchronisation pg 33 to 54
NUMERICALS on Semaphores and critical section pg 54 to 66
DEADLOCK pg 66 to 76
System Calls pg 77 to 82
Process pg 83 to 87
Scheduling pg 88 to 97
I/O pg 94 to 97
Kernel pg 97 to 98

Memory Management

Q1. How can the system distinguish between the pages that are in main memory from the pages that are on the disk?

Q2. Why seek() does not cause disk seek until read/write operation ?

Q3. Explain best fit , first fir and worst fit method.

Q4. What is formula for disk access time?

Q5. Where does VAS reside is it present in the secondary memory or in the ram ?

Q6. What are the differences between storage method of files and directories?

Q7. Consider a system with several running processes. The system is running a modern OS that uses virtual addresses and demand paging. It has been empirically observed that the memory access times in the system under various conditions are: t_1 when the logical memory address is found in TLB cache, t_2 when the address is not in TLB but does not cause a page fault, and t_3 when the address results in a page fault. This memory access time includes all overheads like page fault servicing and logical-to-physical address translation. It has been observed that, on an average, 10% of the logical address accesses result in a page fault. Further, of the remaining virtual address accesses, two-thirds of them can be translated using the TLB cache, while one-third require walking the page tables. Using the information provided above, calculate the average expected memory access time in the system in terms of t_1, t_2 , and t_3 .

Q7. Consider the following line of code in a function of a process. `int *x = (int *)malloc(10 * sizeof(int));` When this function is invoked and executed?

(a) Where is the memory for the variable x allocated within the memory image of the process?

(b) Where is the memory for the 10 integer variables allocated within the memory image of the process?

Q9) What is the need for segmentation in OS? Does segmentation cause fragmentation?

Q10. Explain Thrashing in brief

Q11. What do you mean by page replacement algorithm? Enlist different page replacement algorithms.

Q12. What is fragmentation? Explain types of fragmentations with suitable example.

Q13. Compare contiguous and non contiguous memory allocation of page

Q14. What is virtual memory?

Q15. Explain Compaction in brief.

Q16. Explain Demand paging with example.

Q17. Differentiate internal and external fragmentation

Q18. Explain segmentation using suitable example in brief .

Q19. Explain non-contiguous memory allocation policies with suitable example. (Refer Section 6.7)

Q20. Compare fixed and variable sized partitioning

Q21. Explain Paging with example.

Q22. What is external fragmentation?

Q23. List the page replacement algorithms and explain any one.

Q24. Explain the types of fragmentation and ways to reduce them.

Q25. Define and explain segmentation advantages

Q26. Explain the types of fragmentation and ways to reduce them.

Q27. Explain the techniques of managing memory using first fit, best fit and worst fit with suitable example.

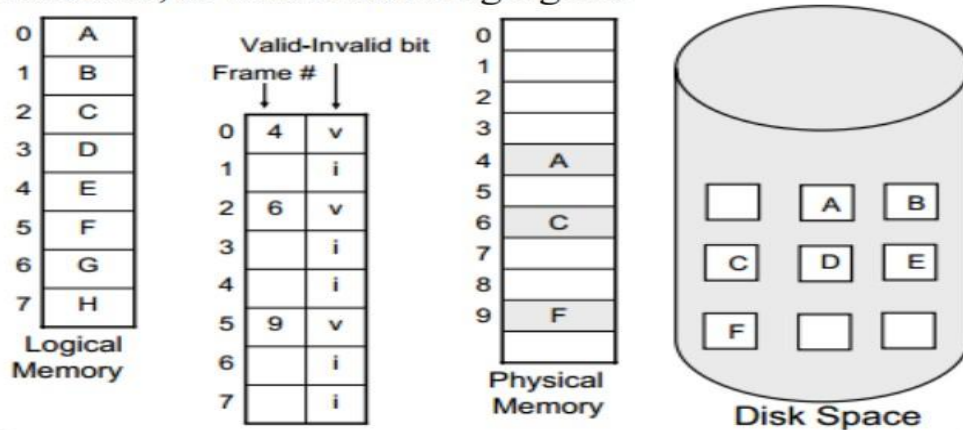
Q28. Explain the concept of segmentation.

Q29. Explain variable size partitioning with its advantages and drawbacks.

Q30. What is swapping? Explain it with suitable example

Q31: How can the system distinguish between the pages that are in main memory from the pages that are on the disk?

Ans: The system uses valid-invalid bit is used. This bit is set to "valid" when the page is in memory, while it is set to "invalid" when the page is either not valid or is the page is valid but is on the disk, as in the following figure.



Q32. What is the difference between paging and segmentation?

Solution :

Paging: It is generally a memory management technique that allows OS to retrieve processes from secondary storage into main memory. It is a non- contiguous allocation technique that divides each process in the form of pages.

Segmentation: It is generally a memory management technique that divides processes into modules and parts of different sizes. These parts and modules are known as segments that can be allocated to process.

Paging	Segmentation
It is invisible to a programmer.	It is visible to a programmer.
In this, the size of pages is fixed.	In this, the size of segments is not fixed.
Procedures and data cannot be separated in paging.	Procedures and data can be separated in segmentation.
It allows a cumulative total of virtual address spaces to cross physical main memory.	It allows all programs, data, and code to break up into independent address spaces.
It is mostly available on CPUs and MMU chips.	It is mostly available on Windows servers that may support backward compatibility, while Linux has limited support.
It is faster for memory access as compared to segmentation.	It is slower as compared to paging.
In this, OS needs to maintain a free frame.	In this, OS needs to maintain a list of holes in the main memory.
In paging, the type of fragmentation is internal.	In segmentation, the type of fragmentation is external.
The size of the page is determined by available memory.	The size of the page is determined by the user.

Q33) Consider a very simple file system for a tiny disk. Each sector on the disk holds 2 integers, and

all data blocks, indirect blocks, and inodes are 1 disk sector in size (each contains 2 integers). All files stored on disk are interpreted as directories by the file system (there are no "data files"). The file system defines the layout for the following data types on disk:

inode = 1 pointer to a data block + 1 pointer to indirect block

indirect block = 2 pointers to data blocks

directory = a regular file containing zero or more pairs of integers; the first integer of each pair is a file name and the second is the file's inumber

The value "99" signifies a null pointer when referring to a disk block address or directory name. An empty directory has one disk block with the contents "99 99".

The inumber for root directory is "/" is 0.

The following data are stored on disk:

inode array:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
10	7		8			3									
6	99		99			99									

disk blocks:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	32		96			1	99	99		57					
	3		1			99	99	99		6					

a. How many entries can appear in a maximum-sized directory? (Each entry is a pair of integers)

Solution: 3 (one data block and two other data blocks pointed by the one indirect block)

b. List all directories stored on this disk (full path names) along with the names of the files stored in each directory.

Solution:

directory path name	inumber	indirect blocks	data blocks	contents (subdirectories)
/	0	6	10 1	/32, /57
/32	3	n/a	8	n/a
/57	6	n/a	3	/57/96
/57/96	1	n/a	7	n/a

c. Modify the above data structures to add an empty directory called "87" to directory "/"

Solution:

directory path name	inumber	indirect blocks	data blocks	contents (subdirectories)
/87	9	n/a	14	n/a
/	0	6	10 1 13	/32, /57, /87

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
10	7		8			3			14						
6	99		99			99			99						

disk blocks:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	32		96			1	99	99		57			87	99	
	3		1			13	99	99		6			9	99	

34) What is the strategy behind First Fit Method, is it faster than Best Fit ?

35) The breaking of physical memory in blocks are called segments.

36) The base address of segment is 1000 and the offset is 4bytes what will be the physical address?

37) Why segmentation may have external fragmentation and why paging doesn't?

38) Given segments sizes which are free in continuous patterns 10,12,14,18,16,5,25.

- Find a place for 5kB in free segment using First-Fit.
- Find a place for 16kB in free segment using Best-Fit.
- Find a place for 7kB in free segment using First-Fit.
- Find a place for 9kB in free segment using Worst-Fit.
- Find a place for 11kB in free segment using Best-Fit.
- Find a place for 4kB in free segment using Worst-Fit.

39) Given 20 milliseconds as the average page-fault service time and a memory access time is 75 nanosecond. What will be the Effective Access Time?

40) Which mapping is stored in Page Table?

41) Assume that there is a main memory and the number of page frames are 6 and the following sequence of page references: 6,4,1,2,7,5,9,4,7,8,9,1,2,6.

a) With FIFO how many number of Page Faults will be there and what will be number of hits?

b) With LRU how many number of Page Faults will be there and what will be number of hits?

42) What will be the number of page faults when the reference string is 7,5,1,2,4,7,5,2 and the number of frames are 5 when LRU is used?

43) What will be the number of page faults when the reference string is 6,5,1,4,2,1,2 and the number of frames are 7 when FIFO is used?

44) State difference between Virtual Address and Physical Address.

45) Given Page Size as 16 bytes, Offset Bits are 4 and Virtual Address requires 11 Bits, what are the number of pages?

46) Given Page Size as 1 KB, Offset Bits are 5 and VPN requires 11 Bits, what are the number of pages?

47) What will be the number of hits and miss when the reference string is 6,5,4,2,4,2,5 and the number of frames are 5 when FIFO is used?

48) When the Pages and Frames are used how the the address is used to find the page number and frame number ?

49) Compare contiguous and non-contiguous memory allocation.

50) Explain in brief about virtual memory.

51) Explain paging with an example.

52) Write all page replacement policies and explain about them.

53) Calculate the number of page faults by using FIFO for page sequence 2,3,2,1,5,2,4,5,3,2,5. The number of frames for this question are 3

54) What is the use of valid-invalid bits in Paging?

55) What is swapping and what is its purpose?

56) State all columns present in the page table.

57) Disadvantages of dynamic relocation.

58) Which one of the following statements about FIFO scheduling is true?

- It is a preemptive scheduling policy
- It reduces average turnaround time
- It reduces average response time
- It suffers from the “convoy” problem
- It is different from the FCFS policy

59) Which one of the following is true about this approach?

- It enables sharing of code between processes
- It translates addresses more slowly than most other approaches
- It provides protection between processes
- It prevents internal fragmentation

60) When allocating a physical page to a process, DOS selects a random free physical page (instead of, for example, the first free one on a free list). This approach will:

- Make the page table larger..
- Make heap allocation fail more often.
- Make physical memory more fragmented externally.
- Slow down address translation.
- None of the above

61) TLB is

- A memory to store page table entries
- A collection of base/bounds pairs to help segmentation work quickly
- An OS component to speed up translation
- A hardware feature that ensures fair use of memory
- An address-translation cache

62) “Data is encoded on each surface in concentric circles of sectors; we call one such concentric circle a _____.”

- platter
- surface
- track
- sector
- block

63) What is a base register for?

64) What type of address is in the base register: physical, or virtual?

65) What is a bounds register for?

66) Why do we need more than one base/bounds register pair?

67) State true or False.

- a. The main reason to have a multi-level page table is to speed up address translation.
- b. The main reason to have a hardware TLB is to speed up address translation.
- c. Using a multi-level page table increases TLB hit time.
- d. As the amount of addressable physical memory grows, the size of each page-table entry (PTE) must grow as well.
- e. As the size of the virtual address space grows, the amount of space occupied by a linear page table also grows.
- f. In a 2-level multi-level page table, each page directory entry should have a valid bit.

68) Name and describe two advantages that segmentation has over simple dynamic relocation

69) In a traditional single-threaded process, What are the stack and heap used for?

70) What are conditions

- TLB Hit
- TLB Miss
- Page Fault

71) State whether one level page table is sufficient for virtual to physical translation.

Provide a valid reason for the same.

Given:

☐ **Virtual Address: 32bit**

☐ **Page size: 1KB page size**

Ans: No it is not possible to maintain a one level page table for virtual to physical translation

Reason: the large memory overhead in maintaining page tables

Explanation:

Size of page table =

(total number of page table entries) * (size of a page table entry)

Let us see how many entries are there in page table

Number of entries in page table =

(virtual address space size) / (page size)

$= (2^{32}) / (2^{10})$

$= 2^{22}$

Now, let us see how big each entry is.

If size of physical memory is 512 MB then number of bits required to address a byte in 512 MB is 29. So, there will be $(512\text{MB}) / (1\text{KB}) = (2^{29}) / (2^{10})$ page frames in physical memory.

To address a page frame 19 bits are required. Therefore, each entry in page table is required to have 19 bits.

Note that page table entry also holds auxiliary information about the page such as a present bit, a dirty or modified bit, address space or process ID information, amongst others. So size of page table

$> (\text{total number of page table entries}) * (\text{size of a page table entry})$

$> (2^{22} * 19) \text{ bytes}$

$> 9.5 \text{ MB}$

And this much memory is required for each process because each process maintains its own page table. Also, size of page table will be more for physical memory more than 512MB.

Therefore, it is advised to use multilevel page table for such scenarios.

72) Given six memory partitions of 300 KB , 600 KB , 350 KB , 200 KB , 750 KB , and 125KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB , 500 KB , 358 KB , 200 KB , and 375 KB (in order)

Ans:

In the First Fit algorithm the First available Memory is allotted to the Process.

In the Best Fit algorithm the Memory in which the least amount of size will be wasted or left over will be allotted.

In the Worst Case Algorithm the Largest Memory will be allotted.

Worst Fit Algorithm : 115 KB is put in 750 KB partition, leaving (300 KB, 600 KB, 350 KB, 200 KB, 635 KB, 125 KB) 500 KB is put in 635 KB partition, leaving (300 KB, 600 KB, 350 KB, 200 KB, 135 KB, 125 KB) 358 KB is put in 600 KB partition, leaving (300 KB, 242 KB, 350 KB, 200 KB, 135 KB, 125 KB) 200 KB is put in 350 KB partition, leaving (300 KB, 242 KB, 150 KB, 200 KB, 135 KB, 125 KB) 375 KB has to wait as no space is available which is having 375 KB of Free Memory.

First Fit Algorithm : 115 KB is put in 300 KB partition, leaving (185 KB, 600 KB, 350 KB, 200 KB, 750 KB, 125 KB) 500 KB is put in 600 KB partition, leaving (185 KB, 100 KB, 350 KB, 200 KB, 750 KB, 125 KB) 358 KB is put in 750 KB partition, leaving (185 KB, 100 KB, 350 KB, 200 KB, 392 KB, 125 KB) 200 KB is put in 350 KB partition, leaving (185 KB, 100 KB, 150 KB, 200 KB, 392 KB, 125 KB) 375 KB is put in 392 KB partition, leaving (185 KB, 100 KB, 150 KB, 200 KB, 17 KB, 125 KB)

Best Fit Algorithm : 115 KB is put in 125 KB partition, leaving (300 KB, 600 KB, 350 KB, 200 KB, 750 KB, 10 KB) 500 KB is put in 600 KB partition, leaving (300 KB, 100 KB, 350 KB, 200 KB, 750 KB, 10 KB) 358 KB is put in 750 KB partition, leaving (300 KB, 100 KB, 350 KB, 200 KB, 392 KB, 10 KB) 200 KB is put in 200 KB partition, leaving (300 KB, 100 KB, 350 KB, 0 KB, 392 KB, 10 KB) 375 KB is put in 392 KB partition, leaving (300 KB, 100 KB, 350 KB, 0 KB, 17 KB, 10 KB)

73) Find

a. How many bits are there in the logical address?

b. How many bits are there in the physical address?

Given: logical address space of 64 pages of

1,024 words each, mapped onto a physical memory of 32 frames.

Ans: Addressing within a 1024-word page requires 10 bits because $1024 = 2^{10}$. Since the logical address space consists of $64 = 2^6$ pages, the logical addresses must be $10 + 6 = 16$ bits. Similarly, since there are $32 = 2^5$ physical frames, physical addresses are $5 + 10 = 15$ bits long.

74) Explain Belady's Anomaly and also state the reasons behind Belady's Anomaly?

Ans:

Belady's Anomaly is the phenomenon of increasing the number of page faults on increasing the number of frames in main memory.

Reason Behind Belady's Anomaly-

An algorithm suffers from Belady's Anomaly if and only if it does not follow stack property. Algorithms that follow stack property are called as stack based algorithms. Stack based algorithms do not suffer from Belady's Anomaly. This is because these algorithms assign priority to a page for replacement that is independent of the number of frames in the main memory. Reason Behind Belady's Anomaly-An algorithm suffers from Belady's Anomaly if and only if it does not follow stack property. Algorithms that follow stack property are called as stack based algorithms. Stack based algorithms do not suffer from Belady's Anomaly. This is because these algorithms assign priority to a page for replacement that is independent of the number of frames in the main memory.

75) Race conditions are possible in many computer systems. Consider a banking system that maintains an account balance with two functions: deposit(amount) and withdraw(amount). These two functions are passed the amount that is to be deposited or

withdrawn from the bank account balance. Assume that a husband and wife share a bank account. Concurrently, the husband calls the withdraw() function, and the wife calls deposit(). Describe how a race condition is possible and what might be done to prevent the race condition from occurring

Ans:

Assume that the balance in the account is \$250.00 and that the husband calls withdraw(\$50) and the wife calls deposit(\$100). Obviously, the correct value should be \$300.00. Since these two transactions will be serialized, the local value of the balance for the husband becomes \$200.00, but before he can commit the transaction, the deposit(100) operation takes place and updates the shared value of the balance to \$300.00. We then switch back to the husband, and the value of the shared balance is set to \$200.00—obviously an incorrect value

76) Given that the time to service a page fault is on the average 10 milliseconds, while a memory access takes 1 microsecond. Then a 99.99% hit ratio results in average memory access time of

Ans: Average memory access time =

$$[(\% \text{ of page miss}) * (\text{time to service a page fault}) + (\% \text{ of page hit}) * (\text{memory access time})] / 100$$

$$(99.99 * 1 + 0.01 * 10 * 1000) / 100 = (99.99 + 100) / 1000 = 199.99 / 1000 = 1.9999 \mu s$$

77) Consider a counting semaphore S is initialized to 7. Then, 20 P operations and 15 V operations are performed on S. What is the final value of S?

Ans:

Final value of semaphore variable S

$$= 7 - (20 \times 1) + (15 \times 1)$$

$$= 7 - 20 + 15$$

$$= 2$$

78) Give a simple example of a page reference sequence where the first page selected for replacement will be different for the clock and LRU page replacement algorithms. Assume that a process is allocated frames, and the reference string contains page numbers from the set 0,1,2,3

79) How does dynamic loading aid in better memory space utilization?

80) What is Cycle Stealing?

81) What is the principle of locality. How is locality exploited with memory virtualisation.

Ans:

The principle of locality is the tendency of a processor to access same set of memory location over a short period of time. There are two types of locality of reference:

a) Temporal locality

b) Spatial locality

So not complete code is needed at particular of time. This can be implemented using memory virtualisation.

82) How does trap differ from interrupts?

Ans:

Traps are raised by user programs to perform some functionalities while interrupts are raised by hardware to perform immediate action.

Trap triggers trap handler while interrupts triggers interrupt handlers.

Traps may occur only at specific times(synchronous) while interrupts may occur at any time(asynchronous).

83) Consider memory mapping using single level page table. Assuming that the required pages are in the memory. Which case is more feasible:

A) Memory reference time $\Rightarrow$ 150 ns with a 20 ns overhead on hit. Assuming a 90% TLB hit and 10% TLB miss.

B) Memory reference time $\Rightarrow$ 100 ns with a 40 ns overhead on hit. Assuming a 30% TLB hit and 70% TLB miss.

Ans:

A) effective mem access time = $(150 + 20) \times 0.9 + (150 + 150) \times 0.1 = 183$

B) effective mem access time = $(100 + 40) \times 0.3 + (100 + 100) \times 0.7 = 182$

$B < A$: B is more feasible situation.

84) What is the purpose of valid, protection, dirty bits?

85) What happens as you increase the no of pages that are allocated in each address space?

86) Explain why Trashing occurs and how to avoid it?

1) In paging, what happens if linear page table becomes too big? What can be the solutions possible to avoid this?

2) Consider the Pages referenced by the CPU in the order are 6, 7, 8, 9, 6, 7, 1, 6, 7, 8, 9, 1. Do consider that there are 3 frames and perform replacement using FIFO, LRU, Optimal page replacement. Which replacement policy works fine here in this case?

Fragmentation and Paging

QUESTION 1

DESIGNING A MULTI-LEVEL PAGE TABLE

Given a VAS of 'v' bit address, where size of physical memory is 'm' bytes, and a page size of 's' bytes, calculate the number of levels required in a multi-level page table required to hold the mapping of pages. (consider additional 'x' bits required to store other information in a PTE - like valid bit, dirty bit, etc.)

Solution:

number of page frames (f) = m/s , PFN bits =

$\log_2(f)$

therefore, size of one PTE (e) = $x + \log_2(f)$ bits = $(x + \log_2(f))/8$ bytes

max number of PTEs one page can hold = $s/e = s / ((x + \log_2(f))/8)$
= $8s / (x + \log_2(f))$

so, PT index bits = $\log_2(\text{max PTEs on a page}) = \log_2(8s / (x + \log_2(f)))$
= $\log_2(8s / (x + \log_2(m/s)))$...since $f = m/s$

offset = $\log_2(s)$

we have the total levels in a multi-level page table as $(\text{VAS bits} - \text{offset}) / \text{PT index bits}$

hence,

levels in multi-PT designed for given information =

$$\frac{v - \log_2(s)}{\log_2\left(\frac{8s}{x + \log_2\left(\frac{m}{s}\right)}\right)}$$

Q2. When will the page faults occur? What is the procedure for handling the page fault?

Q3. During defragmentation do we need to block all running processes to serve memory requests?

Q4. How many page faults would occur for the FIFO page replacement algorithm, assuming three frames per page? reference string 1 2 3 4 2 1 5 6 2 1 2 3 7 6 3 2 1 2 3

Q5. What are the pros and cons of choosing a small page size?

Q6. What are the pros and cons of choosing a large page size?

Q7. Explain the FIFO, optimal and LRU page replacement algorithm, using following string using 3 frames. 7 0 1 2 0 3 0 4 2 3 1 0 3 1 2 4 1 2

Q8. What is the basic approach of page replacement?

Q9. What is Belady's anomaly? In which algorithm does it occur?

Q10. Count and demonstrate the number of page faults with number of frames 3 with the following strings

7 0 1 2 3 0 4 2 3 5 6 2 1 0 1 7 1

Q11. Explain with a suitable example how external fragmentation problem is avoided and internal fragmentation appears in paging?

Q12. What is basic method of segmentation?

Q13. Is it possible for an address to generate neither of the exceptions you listed in question 29,

but still generate a page fault? If so, how? If not, why not?

Q14. difference between paging and segmentation?

Q15. Difference between coarse segmentation and fine grained segmentation?

Q16. Consider a simple system running a single process. The size of physical frames and logical pages is

16 bytes. The RAM can hold 3 physical frames. The virtual addresses of the process are 6 bits in size. The program generates the following 20 virtual address references as it runs on the CPU: 0, 1, 20, 2, 20, 21, 32, 31, 0, 60, 0, 0, 16, 1, 17, 18, 32, 31, 0, 61. (Note: the 6-bit addresses are shown in decimal here.) Assume that the physical frames in RAM are initially empty and do not map to any logical page. (a) Translate the virtual addresses above to logical page numbers referenced by the process. That is, write down the reference string of 20 page numbers corresponding to the virtual address accesses above. Assume pages are numbered starting from 0, 1, ...

(b) Calculate the number of page faults generated by the accesses above, assuming a FIFO page replacement algorithm. You must also correctly point out which page accesses in the reference string shown by you in part (a) are responsible for the page faults.

(c) Repeat (b) above for the LRU page replacement algorithm.

(d) What would be the lowest number of page faults achievable in this example, assuming an optimal page replacement algorithm were to be used? Repeat (b) above for the optimal algorithm.

Q17. Consider a system where each process has a virtual address space of 2^v bytes. The physical address space of the system is 2^p bytes, and the page size is 2^k bytes. The size of each page table entry is 2^e bytes. The system uses hierarchical paging with l levels of page tables, where the page table entries in the last level point to the actual physical pages of the process. Assume $l \geq 2$. Let v_0 denote the number of (most significant) bits of the virtual address that are used as an index into the outermost page table during address translation.

(a) What is the number of logical pages of a process?

(b) What is the number of physical frames in the system?

(c) What is the number of PTEs that can be stored in a page?

(d) How many pages are required to store the innermost PTEs?

(e) Derive an expression for l in terms of v , p , k , and e . (f) Derive an expression for v_0 in terms of l , v , p , k , and e .

(f) Derive an expression for v_0 in terms of l , v , p , k , and e .

Q18. Consider a system with 8-bit virtual and physical addresses, and 16 byte pages. A process in this system has 4 logical pages, which are mapped to 3 physical pages in the following manner: logical page 0 maps to physical page 6, 1 maps to 3, 2 maps to 11, and logical page 5 is not mapped to any physical page yet. All the other pages in the virtual address space of the process are marked invalid in the page table. The MMU is given a pointer to this page table for address translation. Further, the MMU has a small TLB cache that stores two entries, for logical pages 0 and 2. For each virtual address shown below, describe what happens when that address is accessed by the CPU.

Specifically, you must answer what happens at the TLB (hit or miss?), MMU (which page table entry is accessed?), OS (is there a trap of any kind?), and the physical memory (which physical address is accessed?). You may write the translated physical address in binary format. (Note that it is not implied that the accesses below happen one after the other; you must solve each part of the question independently using the information provided above.)

(a) Virtual address 7

(b) Virtual address 20

(c) Virtual address 70

(d) Virtual address 80

Q19. In a demand paging system, it is intuitively expected that increasing the number of physical frames will naturally lead to a reduction in the rate of page faults. However, this intuition does not hold for some page replacement policies. A replacement policy is said to suffer from Belady's anomaly if increasing the number of physical frames in the system can sometimes lead to an increase in the number of page faults. Consider two page replacement policies studied in class: FIFO and LRU. For each of these two policies, you must state if the policy can suffer from Belady's anomaly (yes/no). Further, if you answer yes, you must provide an example of the occurrence of the anomaly, where increasing the number of physical frames actually leads to an increase in the number of page faults. If you answer no, you must provide an explanation of why you think the anomaly can never occur with this policy. Hint: you may consider the following example. A process has 5 logical pages, and accesses them in this order: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. You may find this scenario useful in finding an example of Belady's anomaly. Of course, you may use any other example as well.

Q20. Given a system with four page frames, the following table indicates page, load time, last reference time.

Page	Load Time	Last Reference
0	167	374
1	321	321
2	254	306
3	154	331

Which pages will replace by using FIFO and LRU respectively, when a new page will come?

- a)3,0
- b)1,0
- c)3,2
- d)1,2

Q21. Consider a paging system uses 1 level page table residing in M/M and a TLB M/M. Access time 100 ns. TLB lookup takes 20 ns. Each page transfer takes 500 ns to /from disk. TLB hit ratio 95%, page fault rate=10%. Assume that for 20% of total page fault, a dirty page has to be written back to disk before required page is read in from disk. TLB update time is negligible. Find the average memory access time?

Ans) 155

Q22. Consider a process running on a system with a 52-bit CPU (i.e., virtual addresses are 48 bits in size). The system has a physical memory of 8GB. The page size in the system is 4KB, and the size of a page table entry is 4 bytes. The OS uses hierarchical paging. Which of the following statements is/are true? You can assume $2^{10} = 1K$, $2^{20} = 1M$, and so on.

- (a) We require a 4-level page table to keep track of the virtual address space of a process.
- (b) We require a 5-level page table to keep track of the virtual address space of a process.
- (c) The most significant 9 bits are used to index into the outermost page directory by the MMU during address translation.
- (d) The most significant 40 bits of a virtual address denote the page number, and the least significant 12 bits denote the offset within a page.

Q23. Consider a system with paging-based memory management, whose architecture allows for a 4GB virtual address space for processes. The size of logical pages and physical frames is 4KB. The system has 8GB of physical RAM. The system allows a maximum of 1K (=1024) processes to run concurrently. Assuming the OS uses hierarchical paging, calculate the maximum memory space required to store the page tables of all processes in the system. Assume that each page table entry requires an additional 10 bits (beyond the frame number) to store various flags. Assume page table entries are rounded up to the nearest byte. Consider the memory required for both outer and inner page tables in your calculations.

Q24. A system has 32 bit Virtual address and 8KB page size. main memory is 128 MB. the size of page table, when page table entry has 1 valid bit 1 modified bit and frame bits is ____ (MB). What is the frame entry size?

Q25) what are the problems with paging in OS and does it necessary in OS?

Q26) what are the types in paging? And is there any disadvantages in paging?

Problem 27)

Consider the following page reference string: 7,2,3,1,2,5,3,4,6,7,7,1,0,5, 4,6,2,3,0,1. Assume demand paging with 3 frames, how many page faults would occur for the following replacement algorithms? i. LRU replacement 18 page faults ii. FIFO replacement 17 page faults iii. Optimal replacement 13 page faults

Problem 28)

Consider a computer system with a 32-bit logical address and 4-KB page size. The system supports up to 512-MB of physical memory. i. How many entries will be there in a conventional single-level page table and inverted page table? 7 Entries in conventional single-level page table = $2^{32} / 2^{12} = 2^{20} = 106$ Entries in inverted page table = $2^{29} / 2^{12} = 2^{17} = 128 \times 103$ ii. What will be the memory requirement for storing these tables? Memory for storing single-level page table = number of pages $\times$ number of bits for representing frame = $106 \times 17 \text{ bits} = 106 \times 3 \text{ bytes} = 3 \text{ MB}$ Memory for storing inverted page table = number of frames $\times$ number of bits for representing page = $128 \times 103 \times 20 \text{ bits} = 128 \times 103 \times 3 \text{ bytes} = 384 \text{ KB}$ iii. If a memory reference takes 50 nanoseconds, how long does a paged memory reference take in the context of conventional page table? Access time to a paged memory reference take in the context of conventional page table = $2 \times \text{memory access time} = 2 \times 50 = 100 \text{ ns}$ iv. If we add TLBs, and 75% of all page-table references are found in the TLBs, what is the effective memory reference time? (Assume that finding a page-table entry in the TLBs takes 2 nanoseconds, if the entry is present.) Access time in the context of TLB + Page Table (TLB hit = 75% and TLB miss = 25%) = $(0.75 \times (50+2)) + (0.25 \times (2+50+50)) = 64.5 \text{ ns}$

Consider a user program of logical address of size 6 pages and page size is 4 bytes. The physical address contains 300 frames. The user program consists of 22 instructions a, b, c, . . . u, v. Each instruction takes 1 byte. Assume at that time the free frames are 7, 26, 52, 20, 55, 6, 18, 21, 70, and 90. Find the following?

A) Draw the logical and physical maps and page tables? B) Allocate each page in the corresponding frame? C) Find the physical addresses for the instructions m, d, v, r? D) Calculate the fragmentation if exist?

Answer

Physical map The physical address = page size * frame number + offset The physical address of m = $4 \times 20 + 0 = 80$ The physical address of d = $4 \times 7 + 3 = 31$ The physical address of v = $4 \times 6 + 1 = 25$ The physical address of r = $4 \times 55 + 1 = 221$ The external fragmentation = 0 The internal fragmentation = 2

Q29. Use the three initially empty frames in the following page reference. Using the FIFO technique, locate the page errors, where the page reference sequence: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1?

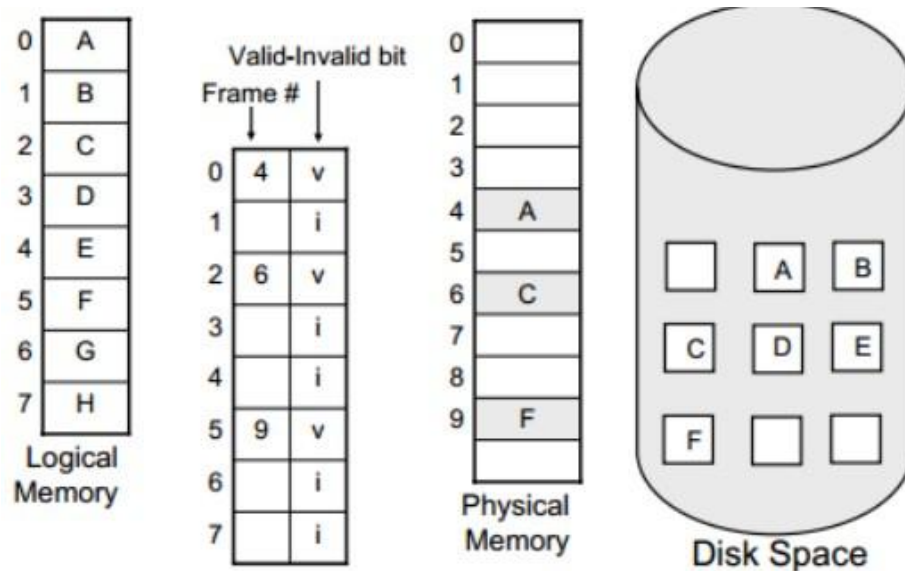
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	0	0	1	2	3	0	4	2	2	2	3	0	0	0	1	2	7
	0	0	1	1	2	3	0	4	2	3	3	3	0	1	1	1	2	7	0
		1	2	2	3	0	4	2	3	0	0	0	1	2	2	2	7	0	2
*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

Page fault

The page fault=15.

Q30. How does the system tell the difference between pages in main memory and pages on the disk?

Ans: The system uses valid-invalid bit is used. This bit is set to "valid" when the page in memory, while it set to "invalid" when the page either not valid or is the page is valid but is on the disk, as in the following figure.



Q31. What is the difference between paging and segmentation?

Answer:

- **Paging:** It is generally a memory management technique that allows OS to retrieve processes from secondary storage into main memory. It is a non-contiguous allocation technique that divides each process in the form of pages.
- **Segmentation:** It is generally a memory management technique that divides processes into

modules and parts of different sizes. These parts and modules are known as segments that can be allocated to process.

Q32. Let us assume , a machine with a 32-bit virtual address space and 64 MB physical memory. What is the approximate size of the page table if the page size is 4KB? (We assume single level paging)

Ans. We know that :

No. of entries in page table = (Virtual address space size)/(page size) So, no. of entries = $2^{(32-12)}$
= 2^{20}

No. of bits required to address the given physical memory = 26 So, number of page frames = $2^{(26-12)} = 2^{14}$

Each page table entry will contain 14 bits address of the page frame and 1 bit for valid-invalid bit.

Page table entry = 16 bits = 2 bytes

Size of page table = (total no. of page table entries) * (size of a page table entry)
= $(2^{20} * 2) = 2\text{MB}$

Q33. Why is paging used?

Ans. Accessing data more quickly can be accomplished through paging. Because the OS copies a set number of pages from the storage device into the main memory, each time a program needs a page, it can find it there. Paging permits a process's physical address space to be noncontiguous.

Q34. Consider the following Page reference string 1,2,3,4,2,3,4,5,6,7,3,2,4. The number of page frames = 4, calculate the page faults and the hit ratio for First In First Out Page replacement algorithm

Q35. Consider page sequence 2,3,2,1,5,2,4,5,3,2,5,2 and discuss working of following page replacement policies. Also count page faults: (use no. of Frames = 3)

Q36. How logical address is converted into physical address by memory management unit? Explain it with example.

Problem-37

Suppose an average page fault service time is 10 milliseconds, while a memory access takes 1 microsecond. Then a 99.99% hit ratio results in average memory access time of _____

Solution:

Average memory access time =

$$[(\% \text{ of page miss}) * (\text{time to service a page fault}) + (\% \text{ of page hit}) * (\text{memory access time})] / 100$$

So, average memory access time in microseconds is

$$(99.99 * 1 + 0.01 * 10 * 1000) / 100 = (99.99 + 100) / 1000 = 199.99 / 1000 = 1.9999 \mu s$$

Problem-38

Consider a user program of logical address of size 6 pages and page size is 4bytes. The physical address contains 300 frames. The user program consists of 22 instructions a, b, c, . . . u, v. Each instruction takes 1 byte. Assume at that time the free frames are 7, 26, 52, 20, 55, 6, 18, 21, 70, and 90. Find the following?

- Draw the logical and physical maps and page tables?
- Allocate each page in the corresponding frame?
- Find the physical addresses for the instructions m, d, v, r?
- Calculate the fragmentation if exist?

Solution :

0	. a . b . c . d
1	. e . f . g . h
2	. i . j . k . l
3	. m . n . o . p
4	. q . r . s . t
5	. u . v

Logical map

Page number	Frame number
0	7
1	26
2	52
3	20
4	55
5	6

page table

	Contents
6	. u . v
7	. a . b . c . d
20	. m . n . o . p
26	. e . f . g . h
52	. i . j . k . l
55	. q . r . s . t

The physical address = page size * frame number + offset
 The physical address of m = $4 * 20 + 0 = 80$
 The physical address of d = $4 * 7 + 3 = 31$ The
 physical address of v = $4 * 6 + 1 = 25$ The physical
 address of r = $4 * 55 + 1 = 221$ The external
 fragmentation = 0
 The internal fragmentation = 2

Problem-39

Consider the following page reference using three frames that are initially empty. Find the page faults using LRU algorithm, where the page reference sequence:

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1?

Solution :

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
	0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
		1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7

Therefore total page faults = 12

Problem-40

A processor uses 36 bit physical addresses and 32 bit virtual addresses, with a page frame size of 4 Kbytes. Each page table entry is of size 4 bytes. A three level page table is used for virtual to physical address translation, where the virtual address is used as follows

- Bits 30-31 are used to index into the first level page table
- Bits 21-29 are used to index into the second level page table
- Bits 12-20 are used to index into the third level page table, and
- Bits 0-11 are used as offset within the page

The number of bits required for addressing the next level page table (or page frame) in the page table entry of the first, second and third level page tables are respectively

- (A) 20, 20 and 20
 (B) 24, 24 and 24
 (C) 24, 24 and 20
 (D) 25, 25 and 24

Solution :

Virtual address size = 32 bits Physical

address size = 36 bits Physical memory

size = 2^{36} bytes

Page frame size = 4K bytes = 2^{12} bytes

No. of bits required to access physical memory frame = $36 - 12 = 24$ So in third level of page table, 24 bits are required to access an entry.

9 bits of virtual address are used to access second level page table entry and size of pages in second level is 4 bytes. So size of second level page table is $(2^9) \times 4 = 2^{11}$ bytes. It means there are $(2^{36}) / (2^{11})$ possible locations to store this page table. Therefore the second page table requires 25 bits to address it. Similarly, the third page table needs 25 bits to address it.

Q41: A system has 32 bit Virtual address and 8KB page size. main memory is 128 MB. the size of page table, when page table entry has 1 valid bit 1 modified bit and frame bits is _____(MB) p.s what is frame bits.. is it the frame entry size?

Ans: By default the address translation of logical (virtual address) into physical address is done by page table.

A physical memory (main memory) page is also known as frame.. Hence number of frames in main memory will decide how many frame bits we require for each page table.. In addition to this we can have other bits in each page table entry like valid bit to determine whether the referred page is in valid address space or not, then dirty bits, protection bits etc are also made part of page table entry sometimes..

But unless mentioned they are not included in page table entry. Only frame no is considered..

We know

Page table size = No of logical pages * Page table entry size (Frame bits + Valid bits + Modified bits)

$$\begin{aligned} \text{So no of pages} &= \text{Virtual address space} / \text{Page size} \\ &= 2^{32} / 2^{13} \\ &= 2^{19} \end{aligned}$$

$$\begin{aligned} \text{No of frame bits} &= \log_2(\text{No of frames}) \\ &= \log_2(2^{19} / 2^{13}) \\ &= 6 \text{ bits} \end{aligned}$$

In addition one valid and one modified bit So page

table entry size = 6 bits + 2 bits = 8 bits

$$\begin{aligned} \text{So page table size} &= \text{No of pages} * \text{Page table entry size} \\ &= 2^{19} * 8 \text{ bits} \\ &= 2^{20} \text{ bits} \\ &= 1 \text{ MB} \end{aligned}$$

Hence size of page table required = 1 MB

Q42: A processor uses 36 bit physical addresses and 32 bit virtual addresses, with a page frame size of 4 Kbytes. Each page table entry is of size 4 bytes. A three level page table is used for virtual to physical address translation, where the virtual address is used as follows

- Bits 30-31 are used to index into the first level page table
- Bits 21-29 are used to index into the second level page table
- Bits 12-20 are used to index into the third level page table, and
- Bits 0-11 are used as offset within the page

The number of bits required for addressing the next level page table (or page frame) in the page table entry of the first, second and third level page tables are?

Ans: 25, 25, 24

Virtual address size = 32 bits
Physical address size = 36 bits
Physical memory size = 2^{36} bytes

Page frame size = 4K bytes = 2^{12} bytes

No. of bits required to access physical memory frame = $36 - 12 = 24$

So in the third level of the page table, 24 bits are required to access an entry.

9 bits of virtual address are used to access the second level page table entry and size of pages in the second level is 4 bytes. So size of second level page table is $(2^9) * 4 = 2^{11}$ bytes. It means there are $(2^{36}) / (2^{11})$ possible locations to store this page table. Therefore the second page table requires 25 bits to address it.

Similarly, the third page table needs 25 bits to address it.

Q43: Explain demand paging?

Ans: Demand paging is a method that loads pages into memory on demand. This method is mostly used in virtual memory. In this, a page is only brought into memory when a location on that particular page is referenced during execution.

The following steps are generally followed:

Attempt to access the page.

If the page is valid (in memory) then continue processing instructions as normal. If a page is invalid then a page-fault trap occurs.

Check if the memory reference is a valid reference to a location on secondary memory. If not, the process is terminated (illegal memory access). Otherwise, we have to page in the required page.

Schedule disk operation to read the desired page into main memory. Restart the instruction that was interrupted by the operating system trap

Q44: List the steps needed to handle page fault.

Ans:

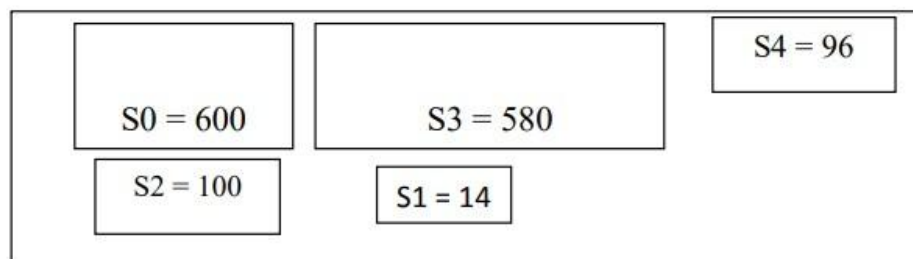
1. The memory address requested is first checked, to make sure it was a valid memory request.
2. If the reference was invalid, the process is terminated. Otherwise, the page must be paged in.
3. A free frame is located, possibly from a free-frame list.
4. A disk operation is scheduled to bring in the necessary page from disk.
(This will usually block the process on an I/O wait, allowing some other process to use the CPU in the meantime.)
5. When the I/O operation is complete, the process's page table is updated with the new frame number, and the invalid bit is changed to indicate that this is now a valid page reference.

6. The instruction that caused the page fault must now be restarted from the beginning, (as soon as this process gets another turn on the CPU.)

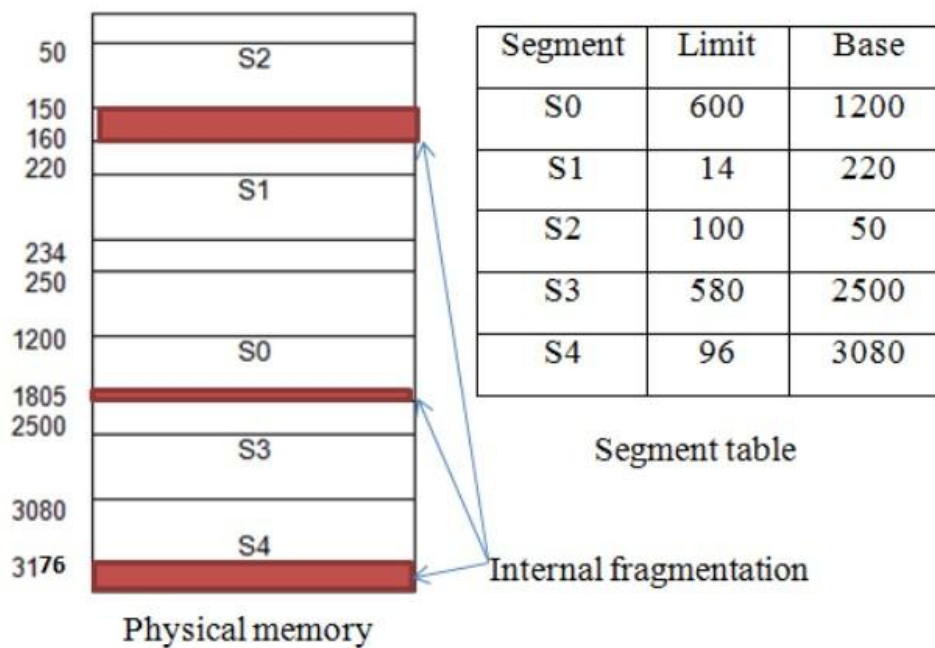
Q45: Consider a program consists of five segments: S0 = 600, S1 = 14 KB, S2= 100 KB, S3 =580 KB, and S4 = 96 KB. Assume at that time, the available free space partitions of memory are 1200–1805, 50 – 150, 220-234, and 2500-3180. Find the following: 1. Draw logical to physical maps and segment table? 2. Allocate space for each segment in memory? 3. Calculate the external fragmentation and the internal fragmentation? 4.What are the addresses in physical memory for the following logical addresses: 0.580, (b) 1.17 (c) 2.66 (d) 3.82 (e) 4.20?

Ans:

The logical map is:



Logical map



External fragmentation = 0.

Internal fragmentation = (160-150) + (1805-1800) + (3180-3176)
= 10 + 5 + 4 = 19

Fragmentation = external fragmentation + internal fragmentation = 0 + 19 = 19

4. The physical addresses are

- a) 0.580 the physical address of 0.580 = $1200 + 580 = 1780$.
- b) 1.17 because $d > \text{limit of } S1$, the address is wrong.
- c) 2.66 the physical address of 2.66 = $50 + 66 = 116$
- d) 3.82 the physical address is of 3.82 is = $2500 + 82 = 2582$
- e) 4.20 the physical address $4.20 = 3080 + 20 = 3100$

Q46) Let us consider a 32-bit machine and subdivide the virtual address into 4 pieces as follows:

8-bit 4-bit 8-bit 12-bit

Assuming 3-level page table is employed, such that the first 8 bits are for the first level and so on. Physical addresses are 44 bits and there are 4 protection bits per page. Answer the following questions .

1. What is the page size in such a system?
2. How much memory is consumed by the page table and wasted by internal fragmentation for a process that has 64K of memory starting at address 0?
3. How much memory is consumed by the page table and wasted by internal fragmentation for a process that has a code segment of 48K starting at address 0x1000000, a data segment of 600K starting at address 0x8000000 and a stack segment of 64K starting at address 0xf000000 and growing upward (towards higher addresses)?

Answer 1:

1. 4K. The last 12 bits of the virtual address are the offset in a page, varying from 0 to 4095. So the page size is 4096, that is, 4K.

Answer 2:

2912 or 4224 bytes for page tables, 0 bytes for internal fragmentation. Using the subdivision above, the 1st level page table contains 256 entries, each entry pointing to a 2nd level page table. A 2nd level page table contains 16 entries, each entry pointing to a 3rd page table. A 3rd page table contains 256 entries, each entry pointing to a page. The process's address space consists of 16 pages, thus we need 1 third-level page table. Therefore we need 1 entry in a 2nd level page table, and one entry in the first level page table. Therefore the size is: 256 entries for the first table, 16 entries for the 2nd level page table, and 1 3rd level page table containing 256 entries.

Since physical addresses are 44 bits and page size is 4K, the page frame number occupies 32 bits. Taking the 4 protection bits into account, each entry of the level-3 page table takes $(32+4) = 36$ bits. Rounding up to make entries byte (word) aligned would make each entry consume 40 (64) bits or 5 (8) bytes. For a 256 entry table, we need 1280 (2048) bytes. The top-level page table should not assume that 2nd level page tables are page-aligned. So, we store full physical addresses there. Fortunately, we do not need control bits. So, each entry is at least 44 bits (6 bytes for byte-aligned, 8 bytes for word-aligned). Each top-level page table is therefore $256 * 6 = 1536$ bytes ($256 * 8 = 2048$ bytes).

Trying to take advantage of the 256-entry alignment to reduce entry size is probably not worth the trouble. Doing so would be complex; you would need to write a new memory allocator that guarantees such alignment. Further, we cannot quite fit a table into a 1024-byte aligned region ($44 - 10 = 34$ bits per address, which would require more than 4 bytes per entry), and rounding the size up to the next power of 2 would not save us any size over just storing pointers and using the regular allocator.

Similarly, each entry in the 2nd level page table is a 44-bit physical pointer, 6 bytes (8 bytes) when aligned to byte (word) alignment. A 16 entry table is therefore 96 (128) bytes. So the space required is 1536 (2048) bytes for the top-level page table + 96 (128) bytes for one second-level page table + 1280 (2048) bytes for one third-level page table = 2912 (4224) bytes. Since the process can fit exactly into 16 pages, there is no memory wasted by internal fragmentation.

Answer 3:

5664 or 8576 bytes for page tables, 0 bytes. First, the stack, data and code segments are at addresses that require having 3 page table entries active in the first level page table, so we have 3 second-level page tables. For 48K, you need 12 pages or 1 third-level page table; for 600K, you need 150 pages, or 1 third-level page table and for 64K you need 16 pages or 1 third-level page table.

So the space required is 1536 (2048) bytes for the top level page table + 3 * 96 (3 * 128) bytes for 3 second-level page tables + 3 * 1280 (3 * 2048) for 3 third-level page table = 5664 (8576) bytes. As the code, data, stack segment of the process fits exactly into 12, 150, 16 pages respectively, there is no memory wasted by internal fragmentation.

Q47) Consider a two level paging scheme. The page size is 4 KB and page table entry size is 4 bytes. The size of outer page table is 4 KB. Give the division of virtual address space.

Sol: Given-

- Page size = 4 KB
- Page table entry size = 4 bytes
- Page table size = 4 KB

Let the number of bits in virtual address = n bits



Number of Bits in Page Offset-

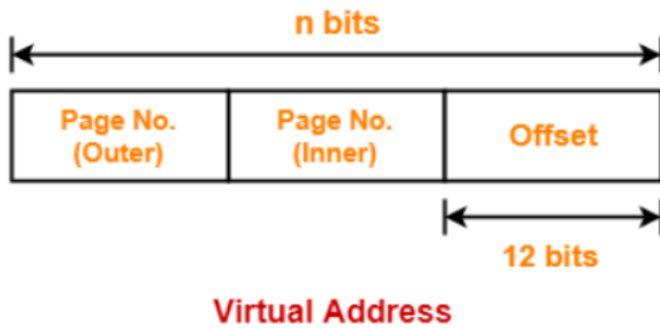
We have,

Page size

= 4 KB

= 212 B

Thus, Number of bits in page offset = 12 bits



Process Size-

Number of bits in virtual address = n bits

Thus,

Process size = 2^n bytes

Number of Pages of Process-

Number of pages the process is divided

= Process size / Page size

= $2^n \text{ B} / 4 \text{ KB}$

= $2^n \text{ B} / 2^{12} \text{ B}$

= 2^{n-12} pages

Inner Page Table Size-

Inner page table keeps track of the frames storing the pages of process.

Inner page table size

= Number of entries in inner page table x Page table entry size

= Number of pages the process is divided x Page table entry size

= $2^{n-12} \times 4 \text{ bytes}$

= 2^{n-12+2} bytes

= 2^{n-10} bytes

Number of Pages of Inner Page Table-

Number of pages the inner page table is divided

= Inner page table size / Page size

= $2^{n-10} \text{ B} / 4 \text{ KB}$

= $2^{n-10} \text{ B} / 2^{12} \text{ B}$

= 2^{n-22} pages

= $2^{n-10-12}$

Now, these 2^{n-22} pages of inner page table are stored in different frames of the main memory.

Number of Page Table Entries in One Page of Inner Page Table-

Number of page table entries in one page of inner page table

= Page size / Page table entry size

= $4 \text{ KB} / 4 \text{ bytes}$

= 1 K

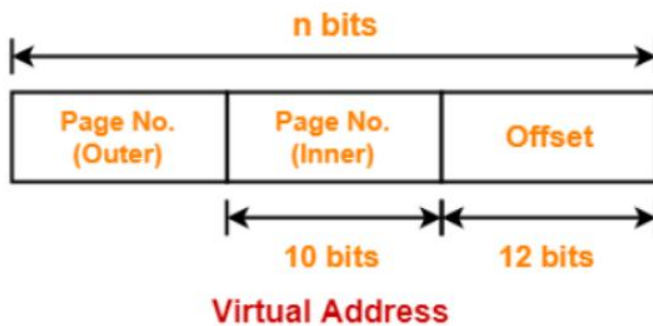
= 2¹⁰ entries

Number of Bits Required to Search an Entry in One Page of Inner Page Table-

One page of inner page table contains 2¹⁰ entries.

Thus,

Number of bits required to search a particular entry in one page of inner page table = 10 bits



Outer Page Table Size-

Outer page table is required to keep track of the frames storing the pages of inner page table.

Outer page table size

= Number of entries in outer page table x Page table entry size

= Number of pages the inner page table is divided x Page table entry size

= $2^{n-22} \times 4$ bytes

= 2^{n-22+2} bytes

= 2^{n-20} bytes

But we are given outer page table size = 4 KB

Thus,

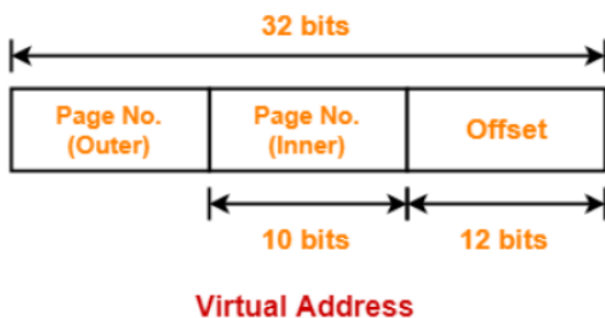
2^{n-20} bytes = 4 KB

$2^{n-20} = 2^{12}$

$n - 20 = 12$

$\therefore n = 32$

Thus, number of bits in virtual address = 32 bits



Number of Bits Required to Search an Entry in Outer Page Table-

Method-01:

Number of bits required to search a particular entry in outer page table

= Number of bits in virtual address – (Number of bits required to search an entry in inner page table

+ Number of bits in page offset)

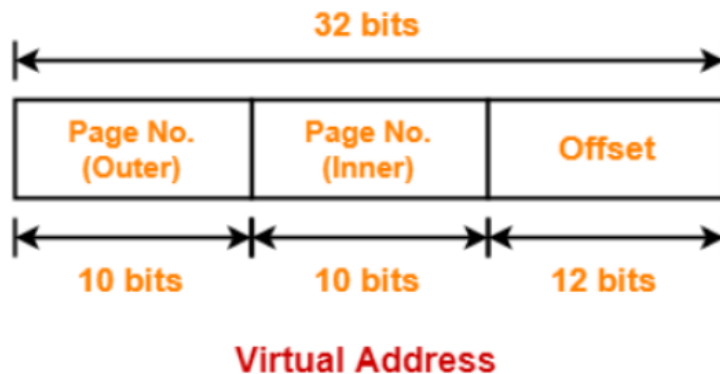
= 32 bits – (10 bits + 12 bits)

= 32 bits – 22 bits
 = 10 bits

Method-02:

Number of entries in outer page table
 = Number of pages the inner page table is divided
 = 2^{n-22}
 = 2^{32-22}
 = 210

Thus, Number of bits required to search a particular entry in outer page table = 10 bits



Q48) Consider a three level paging scheme. The page size is 4 KB and page table entry size is 4 bytes. The size of outermost page table is 4 KB. Give the division of virtual address space.

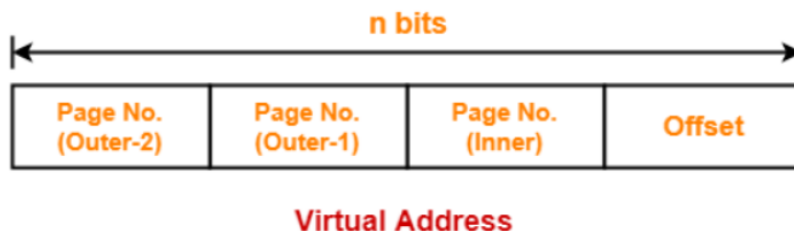
Sol: Consider a three level paging scheme. The page size is 4 KB and page table entry size is 4 bytes. The size of outermost page table is 4 KB. Give the division of virtual address space.

Solution-

Given-

- Page size = 4 KB
- Page table entry size = 4 bytes
- Page table size = 4 KB

Let the number of bits in virtual address = n bits



Number of Bits in Page Offset-

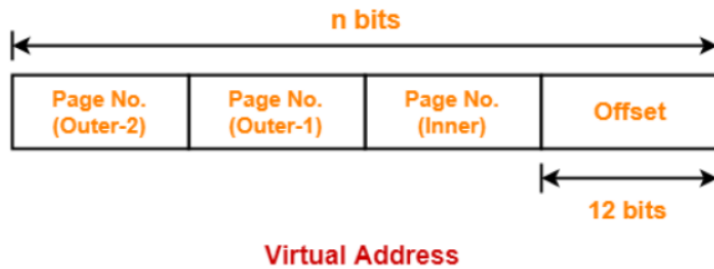
We have,

Page size

= 4 KB

= 212 B

Thus, Number of bits in page offset = 12 bits



Process Size-

Number of bits in virtual address = n bits

Thus,

Process size = 2^n bytes

Number of Pages of Process-

Number of pages the process is divided

= Process size / Page size

= $2^n \text{ B} / 4 \text{ KB}$

= $2^n \text{ B} / 2^{12} \text{ B}$

= 2^{n-12} pages

Inner Page Table Size-

Inner page table keeps track of the frames storing the pages of process.

Inner page table size

= Number of entries in inner page table x Page table entry size

= Number of pages the process is divided x Page table entry size

= $2^{n-12} \times 4 \text{ bytes}$

= 2^{n-12+2} bytes

= 2^{n-10} bytes

Number of Pages of Inner Page Table-

Number of pages the inner page table is divided

= Inner page table size / Page size

= $2^{n-10} \text{ B} / 4 \text{ KB}$

= $2^{n-10} \text{ B} / 2^{12} \text{ B}$

= $2^{n-10-12}$

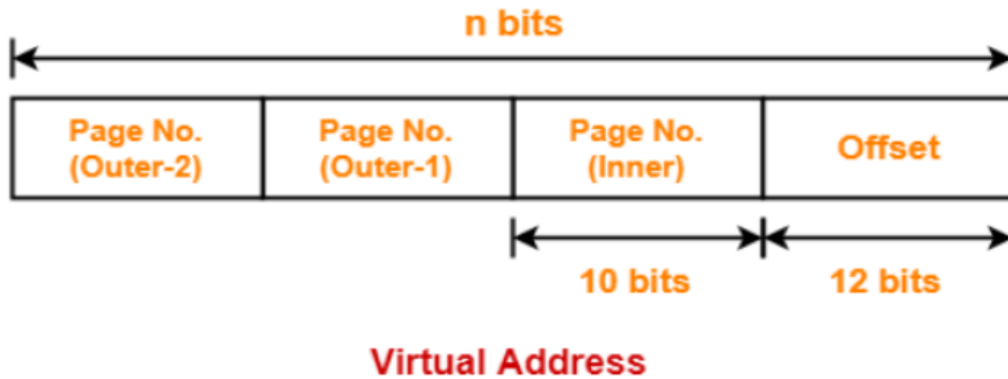
= 2^{n-22} pages

Now, these 2^{n-22} pages of inner page table are stored in different frames of the main memory.

Number of Page Table Entries in One Page of Inner Page Table-

Number of page table entries in one page of inner page table

= Page size / Page table entry size



Outer Page Table-1 Size-

Outer page table-1 is required to keep track of the frames storing the pages of inner page table.

Outer page table-1 size

= Number of entries in outer page table-1 x Page table entry size

= Number of pages the inner page table is divided x Page table entry size

= $2^{n-22} \times 4$ bytes

= 2^{n-22+2} bytes

= 2^{n-20} bytes

Number of Pages of Outer Page Table-1

Number of pages the outer page table-1 is divided

= $4 \text{ KB} / 4 \text{ bytes}$

= 1 K

= 210 entries

Number of Bits Required to Search an Entry in One Page of Inner Page Table-

One page of inner page table contains 210 entries.

Thus, number of bits required to search a particular entry in one page of inner page table = 10 bits

= Outer page table-1 size / Page size

= $2^{n-20} \text{ B} / 4 \text{ KB}$

= $2^{n-20} \text{ B} / 212 \text{ B}$

= $2^{n-20-12}$

= 2^{n-32} pages

Now, these 2^{n-32} pages of outer page table-1 are stored in different frames of the main memory.

Number of Page Table Entries in One Page of Outer Page Table-1

Number of page table entries in one page of outer page table-1

= Page size / Page table entry size

= $4 \text{ KB} / 4 \text{ bytes}$

= 1 K

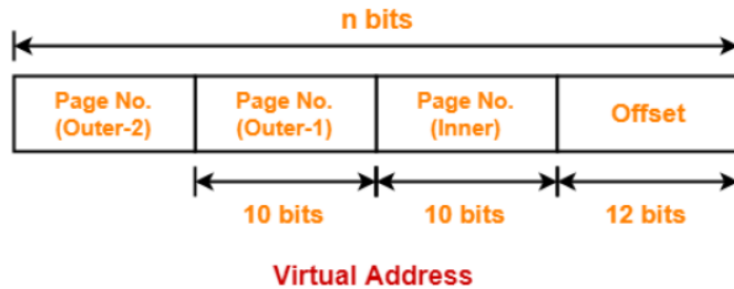
= 210 entries

Number of Bits Required to Search an Entry in One Page of Outer Page Table-1

One page of outer page table-1 contains 210 entries.

Thus,

Number of bits required to search a particular entry in one page of outer page table-1 = 10 bits



Outer Page Table-2 Size-

Outer page table-2 is required to keep track of the frames storing the pages of outer page table-1.

Outer page table-2 size

= Number of entries in outer page table-2 x Page table entry size

= Number of pages the outer page table-1 is divided x Page table entry size

= $2^{n-32} \times 4$ bytes

= 2^{n-32+2} bytes

= 2^{n-30} bytes

But we are given outermost page table size = 4 KB

Thus,

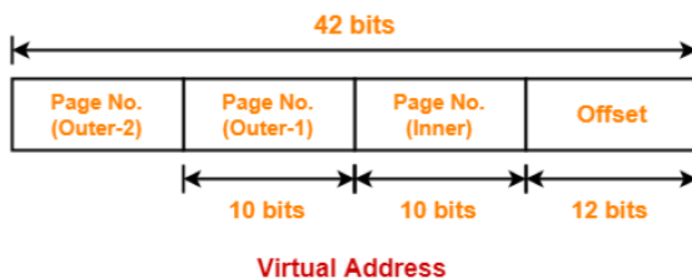
2^{n-30} bytes = 4 KB

$2^{n-30} = 2^{12}$

$n - 30 = 12$

$\therefore n = 42$

Thus, number of bits in virtual address = 42 bits



Number of Bits Required to Search an Entry in Outer Page Table-2

Method-01:

Number of bits required to search a particular entry in outer page table-2

= Number of bits in virtual address – (Number of bits required to search an entry in outer page table-1 + Number of bits required to search an entry in inner page table + Number of bits in page offset)

= 42 bits – (10 bits + 10 bits + 12 bits)

= 42 bits – 32 bits

= 10 bits

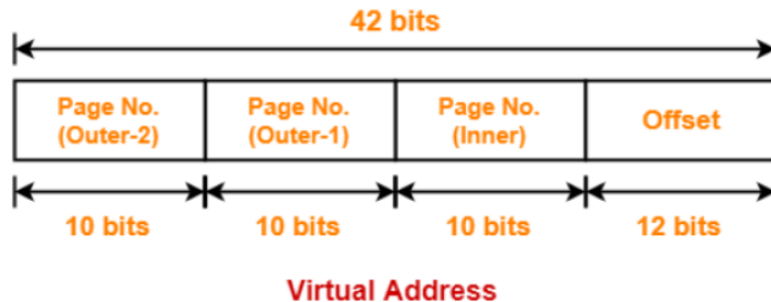
Method-02:

Number of entries in outer page table-2

= Number of pages the outer page table-1 is divided

$$\begin{aligned}
 &= 2^{n-32} \\
 &= 2^{42-32} \\
 &= 2^{10}
 \end{aligned}$$

Thus, Number of bits required to search a particular entry in outer page table-2 = 10 bits



Q49): Calculate the number of address bits, if the memory has a size of 16 GB. Assume the memory is 4-byte addressable.

Sol: If the given address consists of 'k' bits, then 2^k locations are possible.

Size of memory = $2^k \times$ Size of one location.

Given that the size of memory is 16 GB. (16 GB = 2^{34} B)

$$2^k \times 4 = 2^{34}$$

$$2^k = 2^{34} / 2^2 (\because 2^2 = 4)$$

$$2^k = 2^{32}$$

$$\therefore k = 32 \text{ bits}$$

50) State the major difference between External and Internal Fragmentation, give examples in which these type of Fragmentation is shown.

51) What is fragmentation? Explain types of fragmentations with suitable examples.

52) Explain segmentation with example

1) What are the problems in paging that lead to segmentation and what are still persisting even if we use segmentation?

Process Synchronisation

QUESTION 1

CAR DEALERSHIP

A typical car dealership sells and orders cars based on their stock. Once a dealership runs out of cars, it is time to order some cars from the inventory of the car manufacturer, which itself churns out a specific number of cars (a figure predefined based on some supply-demand statistics by a bunch of “has-been” MBA graduates). Your job is to implement a rough algorithm that will oversee this. To keep things simple, we will sell only a generic car model across ‘x’ dealerships...a weird car brand some might say!

Given:

n = dealership stock
m = inventory capacity

Solution:

```
int dealers = x;
```

```
sem_t fullStock[x] = {n} // array of sem init to max cars in stock  
sem_t emptyStock[x] = {0}
```

```
sem_t fullInventory = {m} // sem init to max inventory size  
sem_t emptyInventory
```

```
num_cars[x] = {n}; tot_cars = m;
```

```
buyCars(int c, int d) // c = number of cars customer wants to buy  
                    // d = car dealership number
```

```
{  
    for(i = 0; i < c; i++)  
    {  
        sem_wait(&fullStock[d]);  
        sem_wait(&buyerMutex);  
        purchase(); // complete the purchase of a car  
        num_cars[x]--;  
        sem_post(&buyerMutex);  
        sem_post(&emptyStock[d]);  
    }  
}
```

```
dealer(int d)
```

```
{  
    while(1)  
    {  
        sem_wait(&emptyStock[d]);  
        sem_wait(&buyerMutex);  
        if(d_cars[d] == 0)  
            getCars(d);  
        sem_post(&buyerMutex);  
        // we post on sem fullStock n times to make sure the semantics of our sem (range n) is not violated  
        for(i = 0; i < n; i++)  
        {  
            sem_post(&fullStock[d]);  
        }  
    }  
}
```

```

}
}
}
}
getCars(int d)
{
for(i = 0; i < n; i++)
{
sem_wait(&fullInventory); sem_wait(&sellerMutex); tot_cars--;
num_cars[d]++; sem_post(&sellerMutex); sem_post(&emptyInventory);
}
}
manufacturer()
{
while(1)
{
sem_wait(&emptyInventory); sem_wait(&sellerMutex);
if (tot_cars == 0)
tot_cars += m; // add m cars to inventory sem_post(&sellerMutex);
// we post on sem fullInventory m times to make sure the semantics of our sem (range m) is not
violated
for(i = 0; i < m; i++)
{
sem_post(&fullInventory);
}
}
}

```

Q2 . What is the low level code logic behind the Test and Set atomic implementation Using hardware support?

Q3. How is the lock held by the next thread when `unpark(tid)` system call is used ?

Q4. Write implementation of spin lock using compare and swap instruction.

Q5. What is the solution for race condition.

Q6. What are critical regions? Explain semaphores with example

Q7. Explain spin locks in the context of semaphore .what is solution for race condition?

Q8. . Semaphore itself is a critical section problem .how this can we manage in uniprocessor and multiprocessor condition?

Q9. Describe the classical synchronization problem and solutions for readerswriters problem.

Q10. What is race condition ? why hardware solution of critical sectionsynchronization problem needed?

Q11. What is solution for race condition?

Q12. Explain producer consumer problem.

Q13. Write on Condition Variables.

Q14. While handling a large data structure in a critical section , using a local is efficient or there is any other mechanism to be followed to avoid data race conditions?

Q 15. What happens if a consumer thread in a waiting queue does not respond to the wake signal given by producer thread , is that thread or next thread gets removed from that waiting queue or not?

Q16. What happens if we get into the situation of data race in the lock mechanism itself?

Q17. Consider the following scenario. A town has a very popular restaurant. The restaurant can hold N diners. The number of people in the town who wish to eat at the restaurant, and are waiting outside

its doors, is much larger than N. The restaurant runs its service in the following manner. Whenever it is ready for service, it opens its front door and waits for diners to come in. Once N diners enter, it closes its front door and proceeds to serve these diners. Once service finishes, the backdoor is opened and the diners are let out through the backdoor. Once all diners have exited, another batch of N diners is admitted again through the front door. This process continues indefinitely. The restaurant does not mind if the same diner is part of multiple batches. We model the diners and the restaurant as threads in a multithreaded program. The threads must be synchronized as follows. A diner cannot enter until the restaurant has opened its front door to let people in. The restaurant cannot start service until N diners have come in. The diners cannot exit until the back door is open. The restaurant cannot close the backdoor and prepare for the next batch until all the diners of the previous batch have left. Below is given unsynchronized pseudocode for the diner and restaurant threads. Your task is to complete the code such that the threads work as desired. Please write down the complete synchronized code of each thread in your solution. You are given the following variables (semaphores and initial values, integers) to use in your solution. The names of the variables must give you a clue about their possible usage. You must not use any other variable in your solution. sem (init to 0): entering_diners, exiting_diners, enter_done, exit_done sem (init to 1): mutex_enter, mutex_exit Integer counters (init to 0): count_enter, count_exit All changes to the counters and other variables must be done by you in your solution. None of the actions performed by the unsynchronized code below will modify any of the variables above.

(a) Unsynchronized code for the restaurant thread is given below. Add suitable synchronization in your solution in between these actions of the restaurant. openFrontDoor() closeFrontDoor() serveFood() openBackDoor() closeBackDoor()

(b) Unsynchronized code for the diner thread is given below. Add suitable synchronization in your solution around these actions of the diner. enterRestaurant() eat() exitRestaurant()

Q18. Consider two processes P1 and P2 executing their respective codes. Initial value of global variables P1 and P2 inside are “False”.

```

P1
While(TRUE)
{
    While(P1 inside);
    P1 inside= true;
    C.S.    //Critical Section
    P1 inside=false;
}

```

```

P2
While(TRUE)
{
    While(P2 inside);
    P2 inside= true;
    C.S.    //Critical Section
    P2 inside=false;
}

```

Consider the below statements:

1. Mutual Exclusion is satisfied .
2. Mutual Exclusion is not satisfied
3. It is possible for deadlock.

Which of the above are True?

a) Only 1

- b) Only 2
- c) Only 1 and 3
- d) Only 2 and 3

Answer) b

QQ2. A shared variable x , initialized to zero is operated by four processes W, X, Y, Z. Process W and X increment x by one, while process Y, Z decrement x by two. Each process before reading performs 'wait' on a semaphore 'S' and signal on 'S' after store if semaphore 'S' is initialized to two. Find what is the maximum possible value of x after all processes complete execution?

- a) -2
- b) -1
- c) 1
- d) 2

Answer) d

QQ3. Suppose we want to synchronize two processes P and Q using binary semaphores S, T.

Process P

```
While(1)
{
    W
    Print '0'
    Print '0'
    X
}
```

Process Q

```
While(1)
{
    Y
    Print '1'
    Print '1'
    Z
}
```

Output String should be like: '001100110011'

Which option is correct?

- | | W | X | Y | Z | |
|----|------|------|------|------|----------|
| a) | P(S) | V(S) | P(T) | V(T) | S, T=1 |
| b) | P(S) | V(T) | P(T) | V(S) | S=1, T=0 |
| c) | P(S) | V(T) | P(T) | V(S) | S, T=1 |
| d) | P(S) | V(S) | P(T) | V(T) | S=1, T=0 |

Ans) c

Q19) What is the role of critical section in process synchronization?

Q20) What are the operations available in semaphores? Which kind of problem can be solved using semaphores?

Q21. The atomic fetch-and-set x, y instruction retrieves the previous value of x from y and unconditionally sets the memory location x to 1 without allowing any other access during the process. Take into account the following binary semaphore implementation of the P and V functions.

```
AcquireLock(L){
```

```
    while (Fetch_And_Add(L,1))
```

```
        L = 1;
```

```
}
```

```
ReleaseLock(L){
```

```
    L = 0;
```

```
} This implementation (A) fails as L  
can overflow
```

(B) fails as L can take on a non-zero value when the lock is actually available

**(C) works correctly but may starve some processes (D) works
correctly without starvation**

Answer (B)

Take closer look the below while loop.while

```
(Fetch_And_Add(L,1))
```

```
    L = 1; // A waiting process can be here just after
```

```
    // the lock is released, and can make L = 1.
```

Consider a situation where a process has just released the lock and made $L = 0$. Let there be one more process waiting for the lock, means executing the `AcquireLock()` function. Just after the L was made 0, let the waiting processes executed the line $L = 1$. Now, the lock is available and $L = 1$. Since L is 1, the waiting process (and any other future coming processes) can not come out of the while loop.

The above problem can be resolved by changing the `AcquireLock()` to following.

```
AcquireLock(L){
```

```
    while (Fetch_And_Add(L,1))
```

```
        { // Do Nothing }
```

```
}
```

Q22. The atomic fetch-and-set x, y instruction fetches the previous value of x from y and unconditionally sets the memory location x to 1 without allowing any other access during the process. Take into account the following binary semaphore implementation of the P and V functions.

```
void P (binary_semaphore *s) {unsigned
```

```
    y;
```

```
    unsigned *x = &(s->value);do {
```

```
        fetch-and-set x, y;
```

```

    } while (y);

}

```

void V (binary_semaphore *s) { S->value = 0;

} The implementation may not work if context switching is disabled in P. Explain?

Let us talk about the operation P(). It stores the value of s in x, then it fetches the old value of x, stores it in y and sets x as 1. The while loop of a process will continue forever if some other process doesn't execute V() and sets the value of s as 0. If context switching is disabled in P, the while loop will run forever as no other process will be able to execute V().

Q4 Assume P1 is one of n processes. Pn can reserve and release one resource unit at a time from a pool of m identical ones that they share. The maximum resource requirement of process Pi is Si, where Si > 0.

Which of the following is a necessary condition to prevent deadlock from happening?

Answer (c)

In the extreme condition, all processes acquire Si-1 resources and need 1 more resource. So following condition must be true to make sure that deadlock never occurs.

summation of (Si-1) from i=1 to n < m

The above expression can be written as following summation of (Si) from i=1 to n < (m + n)

Q23. What conditions must be met in order to solve the critical-section problem?

Answer: The requirements are: 1. Mutual exclusion. If process Pi is executing in its critical section, then no other processes can be executing in their critical section. 2. Progress. If no process is executing in its critical section and some processes wish to enter their critical section, then only those processes that are not executing in their remainder sections can participate in the decision on which will enter its CS next, and this selection cannot be postponed indefinitely. (No process should have to wait forever to enter its critical section. 3. Bounded waiting. There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Q24. The P and V operations on counting semaphores are defined as

P(s): s = s-1;
 if s < 0 then wait;
 V(s) : s = s+1;
 if s <= 0 then wakeup a process waiting on s;

Assume that Pb and Vb the wait and signal operations on binary semaphores are provided. Two binary semaphores Xb and Yb are used to implement the semaphore operations P(s) and V(s) as follows:

```

P(s): Pb (Xb);
      S = s - 1;
      if(s < 0){
          Vb(Xb);
          Pb(Yb);
      }

```

```

    }
    Else Vb (Xb); V(s): Pb (Xb);
    S = s + 1;
    if(s <= 0) Vb(Yb); Vb(Xb);

```

The initial values of X_b and Y_b are respectively?

Ans: Explanation: Suppose X_b = 0, then because of P(s): P_b(X_b) operation, X_b will be -1 and process will get blocked as it will enter into waiting section.

So, X_b will be one.

Suppose s=2 (means 2 process are accessing shared resource), taking X_b as 1,

first P(s): P_b(X_b) operation will make X_b as zero. s will be 1 and then V_b(X_b) operation will be executed which will increase the count of X_b as one. Then same process will be repeated making X_b as one and s as zero.

Now suppose one more process comes, then X_b will be 0 but s will be -1 which will make this process go into loop (s < 0) and will result into calling V_b(X_b) and P_b(Y_b) operations. V_b(X_b) will result into X_b as 1 and P_b(Y_b) will result into decrementing the value of Y_b.

case 1: if Y_b has value as 0, it will be -1 and it will go into waiting and will be blocked. total 2 process will access shared resource (according to counting

semaphore, max 3 process can access shared resource) and value of s is -1 means only 1 process will be waiting for resources and just now, one process got blocked. So it is still true.

case 2: if Y_b has value as 1, it will be 0. Total 3 process will access shared resource (according to counting semaphore, max 2 process can access shared resource) and value of s is -1 means only 1 process will be waiting for resources and but there is no process waiting for resources. So it is false.

Q25. A critical section of code must be accessed by two processes, P1 and P2. Think about the processes' use of the following synchronisation construct:

```

/* P1 */ while(true){ want
    s1=true;
    while(wants2 == true){
        /* Critical Section */
        wants1 = false;
    }
    /* Reminder Section */
}
/* P2 */ while(true){ want
    s2=true;
    while(wants1 == true){
        /* Critical Section */
        Wants2 = false;
    }
    /* Reminder Section */
}

```

Here wants1 and wants2 are shared variables, which are initialized to false. Which one of the following statements is TRUE about the above construct?

- (A) It does not ensure mutual exclusion.
- (B) It does not ensure bounded waiting.
- (C) It requires that processes enter the critical section in strict alternation.
- (D) It does not prevent deadlocks, but ensures mutual exclusion. Answer: (D)

Explanation:

Bounded waiting : There exists a bound, or limit, on the number of times other processes are allowed to enter their critical sections after a process has made request to enter its critical section and before that request is granted.

mutual exclusion prevents simultaneous access to a shared resource. This concept is used in concurrent programming with a critical section, a piece of code in which processes or threads access a shared resource.

Q26. what is Process Synchronization and what is the need for it?

Ans-> when multiple processes execute concurrently sharing system resources, then inconsistent results might be produced. Process Synchronization is a mechanism that deals with the synchronization of processes. It controls the execution of processes running concurrently to ensure that consistent results are produced.

Process synchronization is needed-

1. When multiple processes execute concurrently sharing some system resources.
2. To avoid the inconsistent results.

Q27 Consider two processes P1 and P2 are executing concurrently. Both the processes share a common variable named "count" having initial value = 5. Process P1 tries to increment the value of count. Process P2 tries to decrement the value of count. What are the possible values of count?

Ans-> In assembly language, the increment instruction may be written as-

1. LOAD
2. INCREMENT
3. STORE

Case-01: The execution order of the instructions may be-P1(1),

P1(2), P1(3), P2(1), P2(2), P2(3)

In this case, Final value of count = 5

Case-02: The execution order of the instructions may be-P2(1),

P2(2), P2(3), P1(1), P1(2), P1(3)

In this case, Final value of count = 5

Case-03: The execution order of the instructions may be-P1(1),

P2(1), P2(2), P2(3), P1(2), P1(3)

In this case, Final value of count = 6

Case-04: The execution order of the instructions may be-P2(1),

P1(1), P1(2), P1(3), P2(2), P2(3)

In this case, Final value of count = 4

Case-05: The execution order of the instructions may be-P1(1),

P1(2), P2(1), P2(2), P1(3), P2(3)

In this case, Final value of count = 4

Q28 The following two functions P1 and P2 that share a variable B with an initial value of 2 execute concurrently-

```
P1(){ C=B-
```

```
1;B=2*C;
```

```
} P2(){
```

```
D=2*B;
```

```
B=D-1;
```

```
}
```

What are the distinct values of B?

Ans-> Case-01: The execution order of the instructions may be-

P1(1), P1(2), P2(1), P2(2)

In this case, Final value of B = 3

Case-02: The execution order of the instructions may be-P2(1),

P2(2), P1(1), P1(2)

In this case, Final value of B = 4

Case-03: The execution order of the instructions may be-P1(1),

P2(1), P2(2), P1(2)

In this case, Final value of B = 2

Case-04: The execution order of the instructions may be-P2(1),

P1(1), P1(2), P2(2)

In this case, Final value of B = 3

Case-05: The execution order of the instructions may be-P1(1),

P2(1), P1(2), P2(2)

In this case, Final value of B = 3

Case-06: The execution order of the instructions may be-P2(1),

P1(1), P2(2), P1(2)

In this case, Final value of B = 2

From here, Distinct values that may be produced are 2, 3 and 4.

Therefore, Number of distinct values that may be produced = 3

Q29. In the following code,

```
pthread_mutex_t lock =  
PTHREAD_MUTEX_INITIALIZER; pthread_cond_t init  
= PTHREAD_COND_INITIALIZER;  
Pthread_mutex_lock(&lock);  
while (initialized == 0) Pthread_cond_wait(&init, &lock);  
Pthread_mutex_unlock(&lock);
```

The code text to see whether the variable initialized has been set to 0, then calls the wait routine to sleep until the other thread wakes it up.

The following is the awaking code,

```
Pthread_mutex_lock(&lock);  
initialized = 1;  
Pthread_cond_signal(&init);  
Pthread_mutex_unlock(&lock);
```

You might have noticed that the wait call takes a lock as its second input, but the signal call only takes a condition. Explain.

Answer:

- The reason is that the wait call, in addition to putting the calling thread to sleep, releases the lock when putting to sleep the caller.
- Otherwise how could another thread acquire the lock and signal it to wake up?
- However, before returning after being woken, the pthread_cond_wait() re-acquires the lock, thus ensuring that anytime the waiting thread is running between the lock acquired at the beginning of the wait sequence and the lock release at the end, it holds the lock.

Q30. Disabling interrupts was one of the earlier methods proposed for the implementation of locks. Discuss the flaws in this approach.

Answer:

- This approach allows any calling thread to perform a preview of a pre-village operation (turning interrupts on and off), which is quite a risky thing. A greedy program could call lock() at the beginning of its execution and thus monopolize the processor or a malicious program could call lock() and go into an endless loop, and OS will never regain its control

of the system.

This method will not work in multiprocessor scenarios. If multiple threads are running on different CPUs and each tries to enter the same critical section, even if the interrupts are disabled, the threads will be able to run on other processors and can enter the critical section.

- This approach can be inefficient. Compared to normal instruction execution, code that masks or unmask interrupts tends to be executed slowly by modern CPUs.
- For these reasons, turning off interrupts is only used in limited contexts as a solution for mutual exclusion.

Q31. What are the main conditions that an efficient lock is expected to fulfill?

Answer:

- The first and foremost task of a lock is to provide **mutual exclusion** (i.e. preventing multiple threads from entering a critical section).
- To make sure that each thread contending for the lock get a fair shot at acquiring it once it is free. None of the threads **starve**.
- Finally check its **performance**. There are two cases here. First, is, when a single thread is running and grabs and releases the lock, what is the overhead of doing so? Another is the case where multiple threads are contending for the lock on a single CPU.
- By comparing these different scenarios, we can better understand the performance impact of using various locking techniques

Q32. Why can the producer-consumer problem be solved by mutex

Ans. Mutex is used to solve the producer-consumer problem as mutex helps in mutual exclusion. It stops multiple processes from

accessing the critical section. Mutexes only have binary values, 0

and 1, therefore, whenever a process tries to enter the critical section code, it first uses the wait operation to check the mutex value.

Wait(mutex):

Wait(mutex) reduces mutex's value by 1. Consequently, let's say that a process P1 tries to enter the critical region when the mutex value is 1. P1 performs wait(mutex), which lowers the value of mutex. When P1 enters the code's crucial region, the mutex's value now changes to 0.

Now, if Process P2 tries to enter the critical area, it will attempt to lower the mutex value once more. The mutex value, however, is already 0. As a result, wait(mutex) won't run, and P2 will continue to wait for P1 to exit the critical section.

Now, imagine that P2 executes signal to exit the critical section(mutex).

Signal(mutex):

signal(mutex) raises mutex's value by 1. Mutex value once more rises to 1. Now, by executing wait, the process P2 that was in a busy-waiting state will be able to enter the critical section (mutex). Therefore, mutex aids in the processes' mutual exclusion.

Wait and signal operations on mutex are present in the aforementioned section in both the Producer process code and the Consumer process code, which aids in mutual exclusion and fixes the Producer Consumer problem.

Q33. What happens if there is a race condition on the lock itself?

Q34. What happens if data race happen in lock mechanism itself?

Q35. What happens if consumer thread doesn't respond to wake signal by producer thread?

Q36.

Consider the methods used by processes P1 and P2 for accessing their critical sections whenever needed, as given below. The initial values of shared Boolean variables S1 and S2 are randomly assigned.

Method used by P1	Method used by P2
while (S1 == S2);	while (S1 != S2);
Critical Section S1 = S2	Critical Section S2 = !S1

Which one of the following statements describes the properties achieved?

- A) Mutual exclusion but not progress**
- B) Progress but not mutual exclusion**
- C) Neither mutual exclusion nor progress**
- D) Both mutual exclusion and progress**

Solution :

The initial values of shared Boolean variables S1 and S2 are randomly assigned. The assigned values may be-

- S1 = 0 and S2 = 0
- S1 = 0 and S2 = 1
- S1 = 1 and S2 = 0
- S1 = 1 and S2 = 1

Case-01: If S1 = 0 and S2 = 0

In this case,

- Process P1 will be trapped inside an infinite while loop.
- However, process P2 gets the chance to execute.
- Process P2 breaks the while loop condition, executes the critical section and then sets S2 = 1.
- Now, S1 = 0 and S2 = 1.

- Now, process P2 can not enter the critical section again but process P1 can enter the critical section.
- Process P1 breaks the while loop condition, executes the critical section and then sets $S_1 = 1$.
- Now, $S_1 = 1$ and $S_2 = 1$.
- Now, process P1 can not enter the critical section again but process P2 can enter the critical section.

Thus,

- Processes P1 and P2 execute the critical section alternately starting with process P2.
- Mutual exclusion is guaranteed.
- Progress is not guaranteed because if one process does not execute, then other process would never be able to execute again.
- Processes have to necessarily execute the critical section in strict alternation.

Case-02: If $S_1 = 0$ and $S_2 = 1$

In this case,

- Process P2 will be trapped inside an infinite while loop.
- However, process P1 gets the chance to execute.
- Process P1 breaks the while loop condition, executes the critical section and then sets $S_1 = 1$.
- Now, $S_1 = 1$ and $S_2 = 1$.
- Now, process P1 can not enter the critical section again but process P2 can enter the critical section.
- Process P2 breaks the while loop condition, executes the critical section and then sets $S_2 = 0$.
- Now, $S_1 = 1$ and $S_2 = 0$.
- Now, process P2 can not enter the critical section again but process P1 can enter the critical section.

Thus,

- Processes P1 and P2 execute the critical section alternately starting with process P1.
- Mutual exclusion is guaranteed.
- Progress is not guaranteed because if one process does not execute, then other process would never be able to execute again.
- Processes have to necessarily execute the critical section in strict alternation.

Case-03: If $S_1 = 1$ and $S_2 = 0$

This case is same as case-02.

Case-04: If $S_1 = 1$ and $S_2 = 1$

This case is same as case-01.

Thus, Overall we can conclude-

- Processes P1 and P2 executes the critical section alternatively.
- Mutual exclusion is guaranteed.
- Progress is not guaranteed because if one process does not execute, then other process would never be able to execute again.
- Processes have to necessarily execute the critical section in strict alteration.

So option A) is correct

Q37: Let $m[0] \dots m[4]$ be mutexes (binary semaphores) and $P[0] \dots P[4]$ be processes. Suppose each process $P[i]$ executes the following: $\text{wait}(m[i]);$

$\text{wait}(m[(i+1) \bmod 4]);$

$\text{release}(m[i]); \text{release}(m[(i+1) \bmod 4]);$

This could cause

- (a) Thrashing**
- (b) Deadlock**
- (c) Starvation, but not deadlock**
- (d) None of the above**

Ans: Deadlock

Explanation:

You can easily see a deadlock in a situation where..P[0] has acquired $m[0]$ and waiting for $m[1]$
P[1] has acquired $m[1]$ and waiting for $m[2]$ P[2]
has acquired $m[2]$ and waiting for $m[3]$ P[3] has
acquired $m[3]$ and waiting for $m[0]$

Q38: Consider Peterson's algorithm for mutual exclusion between two concurrent processes i and j. The program executed by process is shown below.

repeat
flag[i] = true;

```

turn = j;
while ( P ) do no-op;
Enter critical section, perform actions, then exit criticalsection
flag [ i ] = false;
Perform other non-critical section actions.

```

until false;

For the program to guarantee mutual exclusion, the predicate P in the while loop should be

- a) flag [j] = true and turn = i
- b) flag [j] = true and turn = j
- c) flag [i] = true and turn = j
- d) flag [i] = true and turn = i

3) Assume that n processes, $P_1, \dots, P_n$, share m identical resource units that may

be reserved and released one at a time. Process P_i 's maximum resource need is

S_i , where $S_i > 0$. Which of the following does not qualify as a sufficient requirement for preventing deadlock?

Ans: (b)

Basically, Peterson's algorithm provides guaranteed mutual exclusion by using the two following constructs – flag[] and turn. flag[] controls the willingness of a process to be entered in a critical section. While turn controls the process that is allowed to be entered in the critical section. So by replacing P with the following, flag [j] = true and turn = j process i will not enter the critical section if process j wants to enter the critical section and it is process j's turn to enter the critical section. The same concept can be extended for more than two processes.

Q39) David is a hardware engineer. He came up with a great idea which can help in solving the critical section problem and that he called it as atomic counters. An atomic counter is a variable in memory that can be sampled and incremented in one atomic operation. Also, it can be reset to 0. That is, the two operations allowed on the shared variable a are:

$j = a++$; // execute in one atomic operation $a = 0$;

Show that whether this technique is possible if yes implement it.

Solution:

This hardware solution can help implement a solution to the C.S. as follows:

atomic inside = 0; // inside is an atomic counter

```

int a;
while(true)
{ a = inside++;
  if(a != 1)
    goto retry;
  /* enter C.S. */
  ...
  /* exit C.S. */
  inside = 0;
}

```

Q40: The atomic *fetch-and-set x, y* instruction unconditionally sets the memory location x to 1 and fetches the old value of x in y without allowing any intervening access to the memory location x. consider the following implementation of P and V functions on a binary semaphore .

```
void P (binary_semaphore *s) {
    unsigned y;
    unsigned *x = &(s->value);
    do {
        fetch-and-set x, y;
    } while (y);
}
void V (binary_semaphore *s) {
    S->value = 0;
}
```

Sol: Let us talk about the operation P(). It stores the value of s in x, then it fetches the old value of x, stores it in y and sets x as 1. The while loop of a process will continue forever if some other process doesn't execute V() and sets the value of s as 0. If context switching is disabled in P, the while loop will run forever as no other process will be able to execute V().

Q41: Consider a semaphore S, initialized with value 1. Consider 10 processes P1,P2...P10.All processes have the same code as given below but one process P10 has signal(S) in place of wait(S). If all processes can execute multiple times, then maximum number of processes which can be in the critical section together.

```
While(true) {
    Wait(S)
    C.S
    Signal(s)
}
```

Sol: P1,P2,P3,P4...P9

```
While(true) {
    Wait(S)
    C.S
    Signal(s)
}
```

P10

```
While(true) {
    Signal(S)
    C.S
    Signal(s)
}
```

}

S=1;

Run P10 => S=3

Run P10 => S=5

Run P10 => S=7

Run P10 => S=10

P10 in critical section => S=10

41) In Context Switching between user level threads and kernel level thread, which is faster and why?

42) When the following code is executed, how many child process are created?

```
fork();
```

```
fork();
fork();
fork();
fork();
```

43) When a race condition occurs and what are the possible ways to stop them?

44) What is MUTEX?

45) What are the problems of the synchronizatrion?

46) What are the advantages of the threads over process?

47) What are the advantages of the process over threads?

48) How many processes are created by the following code?

```
Main() {
    Fork();
    Fork();
    Fork();
}
```

49) What are conditional variables and what is the use of it?

50) Write Producer Consumer code.

51) In the dinning philosophers problem there is a round table and N philosophers are sitting with N forks between them. The task given to each philosopher is that he have to pick both the fork on the left and right but with the constraint that if each philosopher first picks the fork an right(lets say) then it will cause deadlock while waiting for the other fork. Write a possible solution for this problem

Ans:

A variable state is associated with each philosopher, and can be one of EATING (holding both forks) or THINKING (when not eating). Further, a condition variable is associated with each philosopher to make them sleep and wake them up when needed. Each philosopher must call the pickup function before eating, and putdown function when done. Both these functions use a mutex to change states only when both forks are available.

bothForksFree(i):

```
return (state[leftNbr(i)] != EATING && state[rightNbr(i)] != EATING)
```

putdown(i):

```
lock(mutex)
```

```
state[i] = THINKING
```

```
if(bothForksFree(leftNbr(i)))
```

```
signal(leftNbr(i))
```

```
if(bothForksFree(rightNbr(i)))
```

```
signal(rightNbr(i))
```

```
unlock(mutex)
```

pickup(i):

```
lock(mutex)
```

```
while(!bothForksFree(i))
```

```
wait(condvar[i])
```

```
state[i] = EATING
```

unlock(mutex)

52) Consider a system consisting of n processes (P0, P1,Pn -1) each process has a segment of code which is known as critical section in which the process may be changing common variable, updating a table, writing a file and so on. The important feature of the system is that when the process is executing in its critical section no other process is to be allowed to execute in its critical section. The execution of critical sections by the processes is a mutually exclusive. The critical section problem is to design a protocol that the process can use to cooperate each process must request permission to enter its critical section. The section of code implementing this request is the entry section. The critical section is followed on exit section. The remaining code is the remainder section

Example:

While (1)

```
{  
Entry Section;  
Critical Section;  
Exit Section;  
Remainder Section;  
}
```

Ans:

A solution to the critical section problem must satisfy the following three conditions.

1. Mutual Exclusion: If process P_i is executing in its critical section then no any other process can be executing in their critical section.
2. Progress: If no process is executing in its critical section and some process wish to enter their critical sections then only those process that are not executing in their remainder section can enter its critical section next.
3. Bounded waiting: There exists a bound on the number of times that other processes are allowed to enter their critical sections after a process has made a request.

53) Multiple people are entering and exiting a room that has a light switch. You are writing a computer program to model the people in this situation as threads in an application. You must fill in the functions onEnter() and onExit() that are invoked by a thread/person when the person enters and exits a room respectively. We require that the first person entering a room must turn on the light switch by invoking the function turnOnSwitch(), while the last person leaving the room must turn off the switch by invoking turnOffSwitch(). You must invoke these functions suitably in your code below. You may use any synchronization primitives of your choice to achieve this desired goal. You may also use any variables required in your solution, which are shared across all threads/persons. (a) Variables and initial values (b) Code onEnter() to be run by thread/person entering (c) Code onExit() to be run by thread/person exiting

Solution:-

variables: mutex, count

onEnter():

lock(mutex)

count++

if(count==1)

```

turnOnSwitch()
unlock(mutex)

```

```

onExit():
lock(mutex)
Count-
if(count==0)
turnOffSwitch()
unlock(mutex)

```

54) Consider the following snapshot of a system running n processes. Process i is holding x_i instances of a resource R for $1 \leq i \leq n$. Currently, all instances of R are occupied. Further, for all i , process i has placed a request for an additional y_i instances while holding the x_i instances it already has. There are exactly two processes p and q such that $y_p = y_q = 0$. Which of the following can serve as a necessary condition to guarantee that the system is not approaching a deadlock?

Solution:-

Processes	P_1	P_2	P_3		P_p		P_q		P_n
Holded Units	x_1	x_2	x_3		x_p		x_q		x_n
Requested Units	y_1	y_2	y_3		0		0		y_n

- Clearly, processes P_p and P_q do not require any additional resource.
- So they continue their execution.
- After getting executed completely, they release the units allocated to them.
- Thus, the total units that get free up $= x_p + x_q$

Now,

- To ensure that other processes are executed without any deadlock, the total amount of units freely available currently $(x_p + x_q)$ must be able to meet the requirements of some other process.
- If available $(x_p + x_q)$ units could not meet the requirement of any other process, then certainly there would be deadlock.

Thus, for no deadlock, the necessary condition is-

$$x_p + x_q \geq \min y_k \text{ where } k \neq p, q$$

55) A single processor system has three resource types X, Y and Z, which are shared by three processes. There are 5 units of each resource type. Consider the following scenario, where the column alloc denotes the number of units of each resource type allocated to each process, and the column request denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish LAST?

Alloc			Request			
	X	Y	Z	X	Y	Z
P0	1	2	1	1	0	3
P1	2	0	1	0	1	2
P2	2	2	1	1	2	0

Solution:-

According to question-

- Total = [X Y Z] = [5 5 5]
- Total _Alloc = [X Y Z] = [5 4 3]

Now,

Available

= Total – Total _Alloc

= [5 5 5] – [5 4 3]

= [0 1 2]

Step-01:

- With the instances available currently, only the requirement of the process P1 can be satisfied.
- So, process P1 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then,

Available

$$= [0 \ 1 \ 2] + [2 \ 0 \ 1]$$

$$= [2 \ 1 \ 3]$$

Step-02:

- With the instances available currently, only the requirement of the process P0 can be satisfied.
- So, process P0 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [2 \ 1 \ 3] + [1 \ 2 \ 1]$$

$$= [3 \ 3 \ 4]$$

Step-03:

- With the instances available currently, the requirement of the process P2 can be satisfied.
- So, process P2 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [3 \ 3 \ 4] + [2 \ 2 \ 1]$$

$$= [5 \ 5 \ 5]$$

Thus,

- There exists a safe sequence P1, P0, P2 in which all the processes can be executed.
- So, the system is in a safe state.
- Process P2 will be executed at last.

56) One example of co-operative approach for switching between processes ?

57) What are types of semaphores and why are they named so?

58) Implement B-tree and start with a simple locking strategy such as a single lock. Measure its performance as the number of concurrent threads increases.

NUMERICALS on Semaphores and critical section

Problem1:

The enter_CS() and leave_CS() functions to implement critical section of a process are realized using test and set instruction as follows-

```
void enter_CS(X)
```

```
{while (test-and-set(X));}
```

```
void leave_CS(X)
```

```
{X = 0;}
```

In the above solution, X is a memory location associated with the CS and is initialized to 0. Now, consider the following statements-

1. The above solution to CS problem is deadlock-free
2. The solution is starvation free
3. The processes enter CS in FIFO order
4. More than one process can enter CS at the same time.

Which of the above statements is true?

1. I only
2. I and II
3. II and III
4. IV only

Solution-

Clearly, the given mechanism is test and set lock which has the following characteristics-

- It ensures mutual exclusion.
- It ensures freedom from deadlock.
- It may cause the process to starve for the CPU.
- It does not guarantee that processes will always execute in a FIFO order otherwise there would be no starvation.

Thus, Option (A) is correct.

Problem2

counting semaphore S is initialized to 10. Then, 6 P operations and 4 V operations are performed on S. What is the final value of S?

Solution-

We know-

- P operation also called as wait operation decrements the value of semaphore variable by 1.
- V operation also called as signal operation increments the value of semaphore variable by 1.

Thus,

Final value of semaphore variable S

$$= 10 - (6 \times 1) + (4 \times 1)$$

$$= 10 - 6 + 4$$

$$= 8$$

Problem3:

A counting semaphore S is initialized to 7. Then, 20 P operations and 15 V operations are performed on S. What is the final value of S?

Solution-

We know-

- P operation also called as wait operation decrements the value of semaphore variable by 1.
- V operation also called as signal operation increments the value of semaphore variable by 1.

Thus,

Final value of semaphore variable S

$$= 7 - (20 \times 1) + (15 \times 1)$$

$$= 7 - 20 + 15$$

$$= 2$$

Problem4:

A shared variable x, initialized to zero, is operated on by four concurrent processes W, X, Y, Z as follows. Each of the processes W and X reads x from memory, increments by one, stores it to memory and then terminates. Each of the processes Y and Z reads x from memory, decrements by two, stores it to memory, and then terminates. Each process before reading x invokes the P operation (i.e. wait) on a counting semaphore S and invokes the V operation (i.e. signal) on the semaphore S after storing x to memory. Semaphore S is initialized to two. What is the maximum possible value of x after all processes complete execution?

1. -2
2. -1
3. 1
4. 2

Solution-

The given question may be pictorially represented as-

Initially, counting semaphore S is initialized with value 2.

Now, We have been asked the maximum possible value of x after all the processes complete execution.

Clearly,

- Processes W and X increments the value of x.
- Processes Y and Z decrements the value of x.

To obtain the maximum value of x, the processes must execute in such a way that-

- Only the impact of the processes W and X remains on the value of x.
- The impact of processes Y and Z gets lost on the value of x.

Step-01:

- Process W arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 1$.
- It reads the value $x = 0$.
- It gets preempted.

Step-02:

- Process Y arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 0$.
- It reads the value $x = 0$.
- It decrements the value of x by 2. Now, $x = 0 - 2 = -2$.
- It writes the value $x = -2$ in the memory.
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 1$.
- Now, execution of process Y is completed.

Step-03:

- Process Z arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 0$.
- It reads the value $x = -2$.
- It decrements the value of x by 2. Now, $x = -2 - 2 = -4$.
- It writes the value $x = -4$ in the memory.
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 1$.
- Now, execution of process Z is completed.

Step-04:

- Process W gets scheduled again.
- It resumes its execution from where it left.
- Before preemption it had already read the value $x = 0$.
- Now, it increments the value of x by 1. Now, $x = 0 + 1 = 1$.
- It writes the value $x = 1$ in the memory.
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 2$.
- Now, execution of process W is completed.

Step-05:

- Process X arrives.
- It executes the wait(S) operation and the value of S decrements by 1. Now, $S = 1$.
- It reads the value $x = 1$.
- It increments the value of x by 1. Now, $x = 1 + 1 = 2$.
- It writes the value $x = 2$ in the memory.
- It executes the signal(S) operation and the value of S increments by 1. Now, $S = 2$.
- Now, execution of process X is completed.

Thus,

- Final value of $x = 2$.
- This is the maximum possible value of x that can be achieved after executing all the 4 processes.
- Option (D) is correct.

Problem5:

Each process P_i , $i = 1, 2, \dots, 9$ is coded as follows-

repeat

P(mutex)

{ Critical Section }

V(mutex)

forever

The code for P_{10} is identical except that it uses V(mutex) in place of P(mutex). What is the largest number of processes that can be inside the critical section at any moment?

1. 1
2. 2
3. 3
4. None of these

Solution-

Code for Processes $P_1, P_2, \dots, P_9$

repeat

P(mutex)

{ Critical Section }

V(mutex)

forever

Code for Process P_{10}

repeat

V(mutex)

{ Critical Section }

V(mutex)

forever

Consider mutex is initialized with 1.

Then-

- All the 10 processes may be present inside the critical section at the same time.
- If there would have been more number of processes, then they could also be present inside the critical section at the same time.

The explanation is as follows-

Explanation-

Scene-01:

- Initially, process P_1 arrives.
- It performs the wait operation on mutex and sets its value to 0.
- It then enters the critical section.

Scene-02:

- Consider the processes $P_2, P_3, \dots, P_9$ arrives when process P_1 is executing the critical section.
- All the other processes get blocked and are put to sleep in the waiting list.

Scene-03:

- Process P_{10} arrives.
- It performs the signal operation on mutex.
- It selects one process from the waiting list say process P_2 and wakes it up.
- Process P_2 can now enter the critical section (P_1 is already present).
- Now, process P_{10} enters the critical section (P_1 and P_2 are already present).
- After executing critical section, during exit, it again performs the signal operation on mutex.
- It selects one process from the waiting list say process P_3 and wakes it up.
- Process P_3 can now enter the critical section (P_1 and P_2 are already present).
- Process P_{10} may keep on executing repeatedly.
- It wakes up 2 processes each time during its course of execution.
- In this manner, all the processes blocked in the waiting list may get entry inside the critical section.

Thus, Option (D) is correct.

Problem6:

Let $m[0] \dots m[4]$ be mutexes (binary semaphores) and $P[0] \dots P[4]$ be processes. Suppose each process $P[i]$ executes the following:

```
wait(m[i]);  
wait(m[(i+1) mod 4]);  
.....  
release(m[i]);  
release(m[(i+1) mod 4]);
```

This could cause-

1. Thrashing
2. Deadlock
3. Starvation but not deadlock
4. None of the above

Solution-

Given processes have to execute the wait operations as-

- Process P_0 : wait(m[0]); wait(m[1]);
- Process P_1 : wait(m[1]); wait(m[2]);
- Process P_2 : wait(m[2]); wait(m[3]);
- Process P_3 : wait(m[3]); wait(m[0]);
- Process P_4 : wait(m[4]); wait(m[1]);

Now,

- A deadlock may be caused when different processes execute the wait operations on mutexes.
- The occurrence of following scenes will give birth to a deadlock.

Scene-01:

- Process P_0 arrives.
- It executes wait(m[0]) and gets preempted.

Scene-02:

Process P_1 gets scheduled.

- It executes wait(m[1]) and gets preempted.

Scene-03:

- Process P_2 gets scheduled.
- It executes wait(m[2]) and gets preempted.

Scene-04:

- Process P_3 gets scheduled.
- It executes wait(m[3]) and gets preempted.

Scene-05:

- Process P_4 gets scheduled.
- It executes $\text{wait}(m[4])$ and gets preempted.

Now,

- The system is in a deadlock state since no process can proceed its execution.
- Thus, Option (B) is correct.

Problem-07:

In the above question, which of the following pairs of processes may be present inside the critical section at the same time?

1. (P_0, P_2)
2. (P_1, P_3)
3. (P_2, P_4)
4. (P_3, P_4)
5. All of these

Solution-

- All the given pair of processes in options execute wait operation on different mutexes.
- Hence, they can get entry inside the critical section at the same time.
- Thus, Option (E) is correct.

NOTE-

- Mutual exclusion is violated here.
- Maximum number of processes that may be present inside the critical section at the same time = 2.

Problem-08:

The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0 = 1$, $S_1 = 0$ and $S_2 = 0$.

Process P0	Process P1	Process P2
<pre>while (true) { wait (S0); print '0' release (S1); release (S2); }</pre>	<pre>wait (S1); release (S0);</pre>	<pre>wait (S2); release (S0);</pre>

How many times will process P0 print '0'?

1. At least twice
2. Exactly twice
3. Exactly thrice
4. Exactly once

Solution-

Maximum Number Of Times Process P0 Can Print '0'-

Maximum number of times process P0 can print '0' = 3
--

The occurrence of following scenes will cause process P0 to print '0' three times-

Scene-01:

- Process P0 arrives.
- It executes wait operation on semaphore S0 successfully. Now, $S0 = 0$.
- It then **prints '0'**. (1st time)
- It executes signal operation on semaphore S1. Now, $S1 = 1$.
- It executes signal operation on semaphore S2. Now, $S2 = 1$.
- While loop causes process P0 to execute again.
- It executes wait operation on semaphore S0 unsuccessfully and gets blocked.

Scene-02:

- Process P1 gets scheduled.
- It executes wait operation on semaphore S1 successfully. Now, $S1 = 0$.
- It executes signal operation on semaphore S0 which wakes up the process P0.
- The execution of process P1 is completed.

Scene-03:

- Process P0 gets scheduled again.
- It **prints '0'**. (2nd time)
- It executes signal operation on semaphore S1. Now, $S1 = 1$.
- It executes signal operation on semaphore S2. Now, $S2 = 1$.
- While loop causes process P0 to execute again.
- It executes wait operation on semaphore S0 unsuccessfully and gets blocked.

Scene-04:

- Process P2 gets scheduled.
- It executes wait operation on semaphore S2 successfully. Now, $S2 = 0$.
- It executes signal operation on semaphore S0 which wakes up the process P0.
- The execution of process P2 is completed.

Scene-05:

- Process P0 gets scheduled again.
- It **prints '0'**. (3rd time)
- It executes signal operation on semaphore S1. Now, S1 = 1.
- It executes signal operation on semaphore S2. Now, S2 = 1.
- While loop causes process P0 to execute again.
- It executes wait operation on semaphore S0 unsuccessfully and gets blocked.

Now,

- The execution of processes P1 and P2 is already completed.
- There is no other process in the system which can perform signal operation on semaphore S0.
- Thus, process P0 can not execute any more.

Thus, maximum number of times process P0 can print '0' = 3 times.

Problem-9

A counting semaphore S is initialised to 10. Then, 6 P operations and 4 V operations are performed on S. What is the final value of S?

Solution :

We know that

- P operation also called as wait operation decrements the value of semaphore variable by 1.
- V operation also called as signal operation increments the value of semaphore variable by 1.

Thus,

Final value of semaphore variable S

$$= 10 - (6 \times 1) + (4 \times 1)$$

$$= 10 - 6 + 4$$

$$= 8$$

10) How would you improve performance when using semaphores?

Ans:

There are a few ways that you could improve performance when using semaphores:

1. Use a lock-free algorithm – this will allow your program to avoid the overhead of acquiring and releasing locks
2. Use a priority queue – this will ensure that the threads with the highest priority are able to acquire the semaphore first
3. Use a fair semaphore – this will ensure that all threads have a fair chance of acquiring the semaphore, instead of one thread hogging the semaphore and preventing other threads from making progress

11) Consider the readers and writers problem discussed above. Recall that multiple readers can be allowed to read concurrently, while only one writer at a time can access the critical section. Write down pseudocode to implement the functions readLock, readUnlock, writeLock, and writeUnlock that are invoked by the readers and writers to realize read/write locks. You must use only semaphores, and no other synchronization mechanism, in your solution. Further, you must avoid using more semaphores than is necessary. Clearly list all the variables (semaphores, and any other flags/counters you may need) and their initial values at the start of your solution. Use the notation down(x) and up(x) to invoke atomic down and up operations on a semaphore x that are available via the OS API. Use sensible names for your variables.

Solution:-

```
sem lock = 1;
sem writer_can_enter = 1;
int readCount = 0;
readLock:
down(lock)
readCount++
if(readCount == 1)
down(writer_can_enter) //don't coexist with a writer
up(lock)
readUnlock:
down(lock)
readCount--
if(readCount == 0)
up(writer_can_enter)
up(lock)
writeLock:
down(writer_can_enter)
writeUnlock:
up(writer_can_enter)
```

DEADLOCK

- Q1. What is deadlock? Explain deadlock prevention in details.
 Q2. What are the methods for handling deadlock ?
 Q3. What are the necessary conditions for deadlock?
 Q4. How we can avoid Deadlock Avoidance?
 Q5. A System requires 2 units of resources having 3 process the minimum no of units of R such that no deadlock will occur?
 Q6. Distinguish between deadlock and starvation.
 Q7. what are the things which leads a system into deadlock situation?
Q8. What is a safe state and what is its use of deadlock avoidance?

A state is safe if the system can allocate all resources requested by all processes (up to their stated maximums) without entering a deadlock state. System is in safe state if there exists a safe sequence of all processes. Deadlock Avoidance: Ensure that a system will never enter an unsafe state.

Q9. What is Deadlock?

Deadlock is a situation when two or more processes wait for each other to finish and none of them ever finish. Consider an example when two trains are coming toward each other on same track and there is only one track, none of the trains can move once they are in front of each other. Similar situation occurs in operating systems when there are two or more processes hold some resources and wait for resources held by other(s).

Q10

A system is having 3 user processes P1, P2 and P3 where P1 requires 21 units of resource R, P2 requires 31 units of resource R, P3 requires 41 units of resource R. The minimum number of units of R that ensures no deadlock is _____?

Solution :

In worst case,

The number of units that each process holds = One less than its maximum demand

So,

- Process P1 holds 20 units of resource R
- Process P2 holds 30 units of resource R
- Process P3 holds 40 units of resource R Thus,
- Maximum number of units of resource R that ensures deadlock = $20 + 30 + 40 = 90$

Minimum number of units of resource R that ensures no deadlock = $90 + 1 = 91$

Q11. How to prevent deadlock?

Answer:

1. Mutual Exclusion:

When you have sharable resources, there will be no fight for resources, which will prevent deadlock conditions.

2. Hold and Wait:

In this condition, a process holding resources is waiting for other resources. But there are ways to eliminate this condition:

1. When the process beforehand specifies the resources it requires. In this way, the process doesn't have to wait for allocation.
2. By releasing all the resources the process is currently holding and then making a new request.

3. No pre-emption:

Preemption means temporary interruption of program execution. Normally processes can't be preempted. Suppose some process P1 wants a resource R1 held by process P2. This situation will lead to a deadlock state. But to avoid deadlock, the process P2 is preempted, and resource R1 could be released and given to process P1. Then we can prevent deadlock.

4. Circular Wait:

We can eliminate this circular wait condition by giving priority to each resource. A process accesses the resources in increasing order of priority. If a request is made for a low-priority resource (being held by some other resource), the request will be considered invalid.

12) Given a system that has 5 number of identical resources. There are N number of processes competing for them. What will be the minimum value of N that can cause deadlock when a process can request for atmost 3 resources?

13) Consider a system as deadlock free and each process can request for maximum 5 resources. What will be the minimum number of identical resources will be the system have?

14) When the deadlock occurs and what are the different handling method to handle deadlocks?

15) What is deadlock? Explain conditions of occurrence deadlock.

16)

Vaibhav, Suraj and Uday go to a Chinese restaurant. But they go there at 8 pm (Busy time). So, waiter apologize and says he can provide only 2 pairs of chopsticks (i.e., four chopsticks), which need to be share between 3 friends. Now, Vaibhav come up with an idea. He suggests to put all four chopsticks in an empty glass at center table & each diner should obey the following protocol:

```
while ( ! had_enough_to_eat() )
{
    aquire_one_chopstick();
    aquire_one_chopstick();
    eat();
    release_one_chopstick();
    release_one_chopstick();
}
```

Can this plan lead to a deadlock? Explain your answer.

Answer: No. Deadlock cannot occur because there are enough chopsticks to guarantee that at least one of the 3 diners will be able to get the 2 chopsticks that he/she needs. Once that diner finishes, he/she will release their chopsticks for someone else. So, everyone will finish.

17) (Read above problem first)

Suppose, now instead of 3 diners, we have 'D' number of diners. Furthermore, each diner may acquire different number of chopsticks. Let's say 'C' number of total chopstick will satisfy the needs of all diners (e.g., for 4 people (D=4), assuming each require 3 chopsticks, then 'C' will be $C = 3*4=12$).

Now, each dinners protocol is displayed below:

```
int s;
int num_sticks = my_chopsticks_requirement();
while (! had_enough_to_eat())
{
    for (s=0; s < num_sticks; s++)
    {
        acquire_one_chopstick();
    }
    eat();
    for (s=0; s < num_sticks; s++)
    {
        release_one_chopstick();
    }
}
```

what is the smallest number of chopsticks (in terms of C and D), needed to ensure that deadlock cannot occur? Explain your answer.

Ans:

$C-D+1$.

This guarantees that every dinner cat get all but one of chopstick it needs (C-D), with one additional chopstick to guarantee that at least one dinner gets all of the chopstick it needs.

Example, Suppose D=2 and diner_1 require 1 chopstick and diner_2 require 11 chopsticks. Therefore C = 12

So, $C-D+1 = 12-2+1=11$

So maximum chopstick requirement is satisfied.

- 18) To write a C program to implement algorithm for deadlock detection
19) What are the necessary conditions which can lead to a deadlock in a system?

BANKERS ALGO

Problem on bankers algo

A single processor system has three resource types X, Y and Z, which are shared by three processes. There are 5 units of each resource type. Consider the following scenario, where the column alloc denotes the number of units of each resource type allocated to each process, and the column request denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish LAST?

1. P0
2. P1
3. P2
4. None of the above since the system is in a deadlock

	Alloc			Request		
	X	Y	Z	X	Y	Z
P0	1	2	1	1	0	3
P1	2	0	1	0	1	2
P2	2	2	1	1	2	0

Solution-

According to question-

- Total = [X Y Z] = [5 5 5]
- Total _Alloc = [X Y Z] = [5 4 3]

Now,

Available

$$= \text{Total} - \text{Total_Alloc}$$

$$= [5 \ 5 \ 5] - [5 \ 4 \ 3]$$

$$= [0 \ 1 \ 2]$$

Step-01:

- With the instances available currently, only the requirement of the process P1 can be satisfied.
- So, process P1 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then,

Available

$$= [0 \ 1 \ 2] + [2 \ 0 \ 1]$$

$$= [2 \ 1 \ 3]$$

Step-02:

- With the instances available currently, only the requirement of the process P0 can be satisfied.
- So, process P0 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [2 \ 1 \ 3] + [1 \ 2 \ 1]$$

$$= [3 \ 3 \ 4]$$

Step-03:

- With the instances available currently, the requirement of the process P2 can be satisfied.
- So, process P2 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [3 \ 3 \ 4] + [2 \ 2 \ 1]$$

$$= [5 \ 5 \ 5]$$

Thus,

- There exists a safe sequence P1, P0, P2 in which all the processes can be executed.
- So, the system is in a safe state.
- Process P2 will be executed at last.

Thus, Option (C) is correct.

Problem-02:

An operating system uses the banker's algorithm for deadlock avoidance when managing the allocation of three resource types X, Y and Z to three processes P0, P1 and P2. The table given below presents the current system state. Here, the Allocation matrix shows the current number of resources of each type allocated to each process and the Max matrix shows the maximum number of resources of each type required by each process during its execution.

	Allocation			Max		
	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3
P1	3	2	0	6	2	0

P2	2	1	1	3	3	3
-----------	---	---	---	---	---	---

There are 3 units of type X, 2 units of type Y and 2 units of type Z still available. The system is currently in safe state. Consider the following independent requests for additional resources in the current state-

REQ1: P0 requests 0 units of X, 0 units of Y and 2 units of Z

REQ2: P1 requests 2 units of X, 0 units of Y and 0 units of Z

Which of the following is TRUE?

1. Only REQ1 can be permitted
2. Only REQ2 can be permitted
3. Both REQ1 and REQ2 can be permitted
4. Neither REQ1 nor REQ2 can be permitted

Solution-

According to question,

$$\text{Available} = [X \ Y \ Z] = [3 \ 2 \ 2]$$

Now,

$$\text{Need} = \text{Max} - \text{Allocation}$$

So, we have-

	Allocation			Max			Need		
	X	Y	Z	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3	8	4	2

P1	3	2	0	6	2	0	3	0	0
P2	2	1	1	3	3	3	1	2	2

Currently, the system is in safe state.

(It is given in question. If we want, we can check)

Checking Whether REQ1 Can Be Entertained-

- Need of P0 = [0 0 2]
- Available = [3 2 2]

Clearly,

- With the instances available currently, the requirement of REQ1 can be satisfied.
- So, banker's algorithm assumes that the request REQ1 is entertained.
- It then modifies its data structures as-

	Allocation			Max			Need		
	X	Y	Z	X	Y	Z	X	Y	Z
P0	0	0	3	8	4	3	8	4	0
P1	3	2	0	6	2	0	3	0	0
P2	2	1	1	3	3	3	1	2	2

Available

$$= [3 \ 2 \ 2] - [0 \ 0 \ 2]$$

$$= [3 \ 2 \ 0]$$

- Now, it follows the safety algorithm to check whether this resulting state is a safe state or not.
- If it is a safe state, then REQ1 can be permitted otherwise not.

Step-01:

- With the instances available currently, only the requirement of the process P1 can be satisfied.
- So, process P1 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [3 \ 2 \ 0] + [3 \ 2 \ 0]$$

$$= [6 \ 4 \ 0]$$

Now,

- It is not possible to entertain any process.
- The system has entered the deadlock state which is an unsafe state.
- Thus, REQ1 will not be permitted.

Checking Whether REQ2 Can Be Entertained-

- Need of P1 = [2 0 0]
- Available = [3 2 2]

Clearly,

- With the instances available currently, the requirement of REQ1 can be satisfied.
- So, banker's algorithm assumes the request REQ2 is entertained.
- It then modifies its data structures as-

	Allocation			Max			Need		
	X	Y	Z	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3	8	4	2
P1	5	2	0	6	2	0	1	0	0
P2	2	1	1	3	3	3	1	2	2

Available

$$= [3 \ 2 \ 2] - [2 \ 0 \ 0]$$

$$= [1 \ 2 \ 2]$$

- Now, it follows the safety algorithm to check whether this resulting state is a safe state or not.
- If it is a safe state, then REQ2 can be permitted otherwise not.

Step-01:

- With the instances available currently, only the requirement of the process P1 can be satisfied.
- So, process P1 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [1 \ 2 \ 2] + [5 \ 2 \ 0]$$

$$= [6 \ 4 \ 2]$$

Step-02:

- With the instances available currently, only the requirement of the process P2 can be satisfied.
- So, process P2 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [6 \ 4 \ 2] + [2 \ 1 \ 1]$$

$$= [8 \ 5 \ 3]$$

Step-03:

- With the instances available currently, the requirement of the process P0 can be satisfied.
- So, process P0 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [8 \ 5 \ 3] + [0 \ 0 \ 1]$$

$$= [8 \ 5 \ 4]$$

Thus,

- There exists a safe sequence P1, P2, P0 in which all the processes can be executed.
- So, the system is in a safe state.
- Thus, REQ2 can be permitted.

Thus, Correct Option is (B).

3) What is Banker's algorithm and what is it used for?

System Calls

Consider the following code from the provided `fork\_shape.c` file

```
| 1 // fork_shape.c: predict the shape of this set of fork() calls
| 2 #include <stdio.h>
| 3 #include <unistd.h>
| 4 #include <wait.h>
| 5 int main(void) {
| 6         pid_t pid = -1;
| 7         for(int i=0; i<4; i++){
| 8             pid = fork();
| 9             if(pid != 0){
|10                 break;
|11             }
|12         }
|13         waitpid(pid, NULL, 0);
|14         printf("pid = %d, parent pid = %d\n", getpid(), getppid());
|15         return 0;
|16 }
```

- How many total processes will print their PIDs?
- What shape of process hierarchy will this code produce? Draw a picture of it.
- Will the order of printing be consistent or arbitrary? Why?

Q2. How are zombie processes created and how do they reap?

-If the parent process exits before the child process the child process becomes an orphan/zombie process and after termination it is adopted by init process where it eventually reaps.

Q3. Why is it important to reap?

-Because the terminated/dead process exists as a zombie process which keeps consuming the memory

allocated to OS Data Structure. wait() reaps any arbitrary child process but if we want to reap any specific child process, we use *waitpid()* function

Q4. wait() block parent until child terminates. Are there any non-blocking ways to invoke wait? -wait() is always blocking and waitpid() can be blocking or non-blocking Using the WNOHANG option ,which Returns immediately regardless of the child's status it can wait without blocking parent process

int pid = waitpid(child_pid, &status, WNOHANG); // specific child OR

int pid = waitpid(-1, &status, WNOHANG); // any child (P.T.O)

```
1 // impatient_parent.c: demonstrate non-blocking waitpid(),
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <sys/wait.h>
6 #include <unistd.h>
7
8 int main(int argc, char* argv){
9     char *child_argv[] = {"/complain",NULL};
10    char *child_cmd = "complain";
11    printf("PARENT: Junior is about to '%s', I'll keep an eye on him\n",
12          child_cmd);
13    pid_t child_pid = fork();
14
15    // CHILD CODE
16    if(child_pid == 0){
17        printf("CHILD: I'm %d and I'm about to '%s'\n",
18              getpid(), child_cmd);
19        execvp(child_cmd, child_argv);
20    }
21
22    // PARENT CODE
23    int status;
24    int retcode = waitpid(child_pid, &status, WNOHANG); // non-blocking wait
25    if(retcode == 0){ // 0 means child has not exited/changed status
26        printf("PARENT: 0? The kid's not done yet. I'm bored\n");
27    }
28    else{ // child has changed status / exited
29        printf("PARENT: Something happend to junior!\n");
30        if(WIFEXITED(status)){
31            printf("Ah, he Exited with code %d", WEXITSTATUS(status));
32        }
33        else{
34            printf("Junior didn't exit, what happened to him?\n");
35        }
36    }
37    return 0;
38 }
```

Q5. What are system calls? How do they work?

Q6. How many new processes are created by the following code?

```

a. Int main () { If(fork) {
                Fork;
            }
            Fork; fork;
        }
    }

```

Q7. Write flow of handling a system call.

Q8. #include <stdio.h> int main() {

if (fork() && fork()) fork();

printf("hello");

return 0;

}

Print the result?

Q9. Is there a mechanism through which child can communicate with the parent process which is created by using fork() ?

Q10. How does it map the calls made by a process as open(), read(), write() ?

Q11. Consider a multithreaded application server handling requests from clients. Every new request that arrives at the server causes a new thread to be spawned to handle that request. The server can provide service to only one request/thread at a time, and other threads that arrive when the server is busy must wait for service using a synchronization primitive (semaphore or condition variable). In order to avoid excessive waiting times, the server does not wish to have more than N requests/threads in the system (including the waiting requests and any request it is currently serving). You may assume that $N > 2$. Given this constraint, a newly arriving thread must first check if N other requests are already in the system: if yes, it must exit without waiting and return an error value to the client, by calling the function `thr_exit failure()`. This function terminates the thread and does not return. When a thread is ready for service, it must call the function `get service()`. Your code should ensure that no more than one thread calls this function at any point of time. This function blocks the thread for the duration of the service. Note that, while the thread receiving service is blocked, other arriving threads must be free to join the queue, or exit if the system is overloaded. After a thread returns from `get service()`, it must enable one of the waiting threads to seek service (if any are waiting), and then terminate itself successfully by calling the function `thr_exit success()`. This function terminates the thread and does not return. You are required to write pseudocode of the function to be run by the request threads in this system, as per the specification above. Your solution must use only locks and condition variables for synchronization. Clearly state all the variables used and their initial values at the start of your solution.

Q12. Consider the following code:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```

    Int main()
{
    If((fork() && (!fork())) {
    If(fork() || fork()) {
    fork();
    }
    }
    printf("hello");
    return 0;
}

```

How many times "Hello" will be printed?

- a)5
- b)6
- c)7
- d)8

Answer) c

Q13.Consider the following code fragment:

```

if (fork() == 0)
{ a = a + 5; printf("%d,%d\n", a, &a); }
else { a = a -5; printf("%d, %d\n", a, &a); }

```

Let u, v be the values printed by the parent process, and x, y be the values printed by the child process. Which one of the following is TRUE?

- (a) $u = x + 10$ and $v = y$**
 - (b) $u = x + 10$ and $v \neq y$**
 - (c) $u + 10 = x$ and $v = y$**
 - (d) $u + 10 = x$ and $v \neq y$**
- Answer
- (c)

fork() returns 0 in child process and process ID of child process in parent process.

In Child (x), $a = a + 5$

In Parent (u), $a = a - 5$; Therefore x

$= u + 10$.

The physical addresses of 'a' in parent and child must be different. But our program accesses virtual addresses (assuming we are running on an OS that uses virtual memory). The child process

gets an exact copy of parent process and virtual address of 'a' doesn't change in child process. Therefore, we get same addresses in both parent and child.

14) Explain in brief about User Calls and Kernel Calls.

15) Why system calls look like procedure calls?

16) Do all privileged functionalities get exposed to users through system calls?

17) What if you don't join threads using pthread.join()?

18) Can the privileged instruction creating or deleting user account be done using system call ?

19) Which is the first program which runs on system boot ?

20) Why we need two system calls fork and exec ?

21) What happens when a timer interrupt is raised ?

1) Using a basic system call as an example (e.g., getpid or uptime), describe what is normally involved in obtaining the result, from the point when the function in the C library is called to the point where the function returns.

Process AND Process States

Q1. Why does the process gets blocked when its initiates an I/O request?

Q2. Give three reasons for the system to select a new process to run.

Q3. State if the given sequence is possible or not.

Process A is running , I/O complete interrupt arrives. Timer interrupt arrives. Scheduler runs. I/O interrupt handler runs. Process B is running.

Q4. Write the flow of context switch

Q5. Explain When a process directly switches from running state to ready state

.which scheduler is involved in such switching?

Q6. Can a running process after going through I/O can be directed taken to ready state without passing through block state? Why?

Q6. what are the responsibilities of short term scheduler , long term scheduler and medium term scheduler?

Q7. What is the difference between process and program?

Program is a set of instructions written to complete a task.

Process is a program in execution.

Program is a passive/static entity.

Process is an active/dynamic entity.

Program resides in secondary memory.

A process in execution resides in Primary Memory.

Program has a longer life span.

A process has a limited life span.

Q8. What is a pipe?

- A traditional pipe is unnamed and can be used only for the communication of related process. If unrelated processes are required to communicate – named pipes are required.
- It is a pipe whose access point is a file available on the file system. When this file is opened for reading, a process is granted access to the reading end of the pipe. Similarly, when the file is opened for writing, the process is granted access to writing end of the pipe.
- A named pipe is also referred to as FIFO or named FIFO.

Q9. A process is a dynamic object, but a program is a static object. State whether given statement is true or not. If true, then elaborate.

Ans-> Yes, given statement is true. A process is a dynamic object, but a program is a static object. A program is static entity. It contains only code. When program is brought in memory and executed then it becomes a process. A program in execution is called process.

A process is an active entity and a program is a passive entity. Process resides in main memory, but program resides in secondary memory. The span of time for the process is limited, but the span of time for the program is unlimited.

Q10. what are the different states of process and explain the transitions between them (when does a process moves from one state to other)?

Ans-> different states of process:

1. Running: The process is being executed.
2. Ready: The process is waiting to be assigned to a processor(waiting to be scheduled).
3. Blocked: suspended, not ready to run

Apart from the above states there are two other states also:

- a. New: The process is being created.
- b. Terminate: The process has finished

execution. Transitions from one state to other:

1. Ready to Running: When it is time to select a process to run, the operating system move the processes from the ready state to the running state.
2. Running to Ready: When the time slot for the process expires or if an interrupt signal occurs, then the operating system shifts the running process to the ready state.
3. Running to Blocked: A process is put into the waiting state if the process needs anevent to occur, or an I/O device is to read.
4. Blocked to Ready: A process in the blocked state is moved to the ready state when the event for which it has been waiting has completed.

Q11. what are the components of process or explain Process Control Block?

Ans-> The PCB is a central store of information that allows the operating system to locate all the key information about the process. The operating system uses this information and performs the operations in the process.

Components of a process:

Memory: code(instruction for program to execute), stack(static memory) and heap(dynamic memory)

CPU: contains registers, stack pointer, program counter and general purpose registers

Context/state related info: pid, user, open file descriptors, etc

All these information is contained inside PCB of process.

Q12. What is difference between process and thread?Ans.

Processes:-

- When any program is in execution ,it is called process.
- Process takes more time for create, terminate and to contextswitch.
- The execution of other processes will not be impacted if oneprocess is blocked.

- Child processes are unaffected by changes to the parent process.
- Processes do not share data with each other. Threads:-
- A segment of process is called thread.
- Thread takes less time for create, terminate and to context switch.
- All other user-level threads are also blocked if one of those threads is blocked.
- Any changes to the main thread may have an impact on the behaviour of the other threads of the process because all threads of the same process share address space and other resources.
- Data is shared between threads.

Q13. What is difference between multitasking and multiprocessing OS?

Q14: Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle?

- (A) 0%
 (B) 10.6%
 (C) 30.0%
 (D) 89.4%

Ans: 10.6%

Let three processes be p0, p1 and p2. Their execution time is 10, 20 and 30 respectively. p0 spends first 2 time units in I/O, 7 units of CPU time and finally 1 unit in I/O. p1 spends first 4 units in I/O, 14 units of CPU time and finally 2 units in I/O. p2 spends first 6 units in I/O, 21 units of CPU time and finally 3 units in I/O.

idle p0 p1 p2 idle 0 2 9 23 44 47

Total time spent = 47

Idle time = 2 + 3 = 5

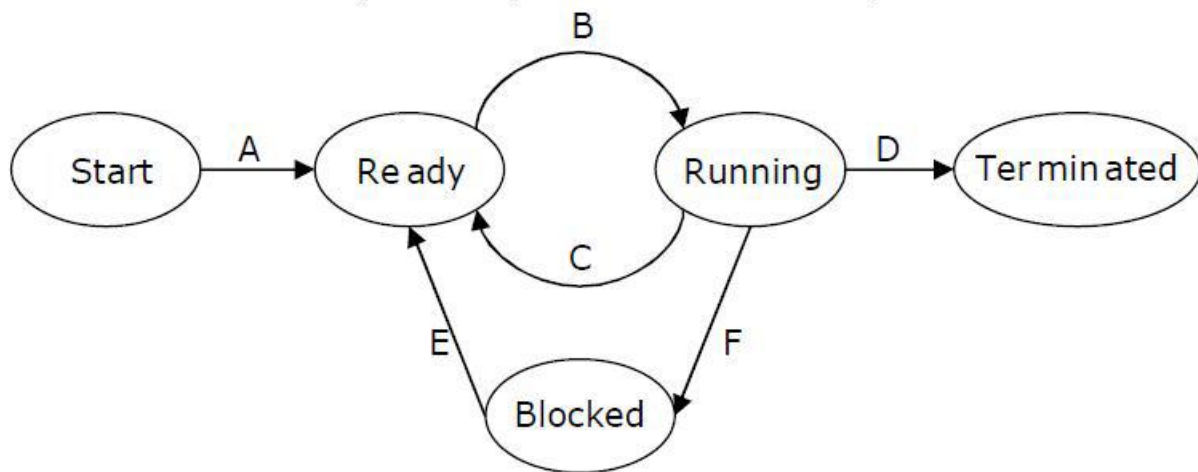
Percentage of idle time = $(5/47) * 100 = 10.6 \%$

15) What makes a process different than a thread?

16) What function is used to create a child process and what function is used to halt the child process?

- 17) How to identify if the current process is child or parent?
- 18) How the fork() function mechanism works?
- 19) How the halting is done is using the wait() function?
- 20) Explain fork(), wait() and exec() relationship.
- 21) What are the different values returned by fork() and what do they stand for?
- 22) Differentiate between process and thread
- 23) What does process control block contain?
- 24) What are the different IPC mechanisms?
- 25) What is main difference between program and process?

Q) Explain with reason which of the given statements are true .Given the process State transition diagram for a uniprocessor system. ASSUMPTION(There is always some process in the ready queue)



- A. A process P2 in blocked state can make transition E while another process P1 is in running state.
- B. The OS uses preemptive scheduling.
- C. If a process makes a transition D, it would result in another process making transition A immediately.
- D. The OS uses non-preemptive scheduling.

Ans:

C is False If a process makes a transition D, it would result in another process making transition B, not A. A is true A process can move to ready state when I/O completes irrespective of other process being in running state or not. B is true

Q. Find the waiting time for the Process P2.

Given:

A) Arrival and execution times of the Process

Process Execution time Arrival time

P1	20	0
P2	25	15
P3	10	30
P4	15	45

B) OS uses SRT (Shortest Remaining Time first Scheduling Algorithm)

Ans:

At time 0, P1 is the only process, P1 runs for 15 time units.

At time 15, P2 arrives, but P1 has the shortest remaining time. So P1 continues for 5 more time units.

At time 20, P2 is the only process. So it runs for 10 time units

At time 30, P3 is the shortest remaining time process. So it runs for 10 time units

At time 40, P2 runs as it is the only process. P2 runs for 5 time units.

At time 45, P3 arrives, but P2 has the shortest remaining time. So P2 continues for 10 more time units.

P2 completes its execution at time 55

Total waiting time for P2 = Completion time - (Arrival time + Execution time)

= 55 - (15 + 25)

= 15

59) Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again.

Assume:

A) All I/O operations can be overlapped as much as possible.

B) The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst

For what percentage of time does the CPU remain idle?

Ans. 10.64

Let three processes be p0, p1 and p2. Their execution time is 10, 20 and 30 respectively. p0 spends first 2 time units in I/O, 7 units of CPU time and finally 1 unit in I/O. p1 spends first 4 units in I/O, 14 units of CPU time and finally 2 units in I/O. p2 spends first 6 units in I/O, 21 units of CPU time and finally 3 units in I/O.

idle p0 p1 p2 idle

0 2 9 23 44 47

Total time spent = 47

Idle time = 2 + 3 = 5

Percentage of idle time = $(5/47) \times 100 = 10.6\%$

Scheduling

Q1. State different scheduling algorithms. Explain them.

Q2. What are the 3 different types of scheduling queues.

Q3. What are the types of scheduling algorithms and explain any of two?

Q4. What is difference between multilevel queue scheduling and multilevel feedback queue scheduling?

Q5. What is response ratio time?

Q6. If there is no pre-emption then which scheduling policies of CPU -Virtualization will be efficient and which gives least response time?

Q7. In a real time system, processes indicate hard constraints about when they need to run. If you were designing a scheduler for such a system, speculate about how would you use that information and what scheduling algorithm you would implement?

Q8. disadvantages of completely fair scheduler?

Q9) Which scheduling policy is most suitable in real time environment? And Which scheduling is fastest?

Q10) Why process scheduling is required and Which type of scheduling is best?

Q11) What is the main purpose of disk scheduling and ,How is performance measured in disk scheduling?

Q12. What are turn around time and response time?

The basic difference b/w the turn around time and response time is:-

- Turn around time is the interval between the submission of a job and its completion.
- Response time is the interval b/w the submission of the request, and the first response to that request.

Problem 13 -

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The C-SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution

Total head movements incurred while servicing these requests

$$= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (199 - 0) + (14 - 0) + (41 - 14)$$

$$= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 27$$

$$= 386$$

Problem 14

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution-

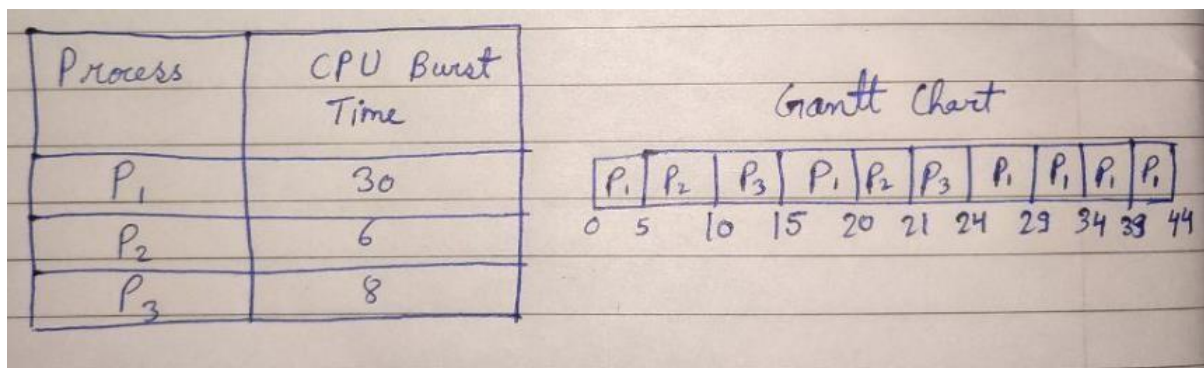
Total head movements incurred while servicing these requests

$$= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (199 - 41) + (41 - 14)$$

$$= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 158 + 27$$

$$= 331$$

Q15. Given a gantt chart and scheduling algorithm used is Round Robin. Time Quantum used is 5ms.



Find the average turnaround time and average waiting time?

Ans-> average turnaround time: we have to find out the turnaround time of each process in order to calculate average turnaround time. Therefore, turnaround time of

$$P_1 = 14 + 30 =$$

$$44\text{ms}$$

$$P_2 = 15 + 6 =$$

$$21\text{ms}$$

$$P_3 = 16 + 8 = 24\text{ms}$$

Average waiting time: Same concept for finding the Waiting Time. Therefore, waiting time of

$$P_1 = 0 + (15 - 5) + (24 - 20) = 14\text{ms}$$

$$P_2 = 5 + (20 - 10) = 15\text{ms}$$

$$P_3 = 10 + (21 - 15) = 16\text{ms}$$

$$\text{Therefore, Average Waiting Time} = (14 + 15 + 16) / 3 = 15\text{ms}$$

Q16. explain the difference between Preemptive Scheduling and Non-Preemptive Scheduling. Give examples of scheduling policies for each of them.

Ans-> In Preemptive Scheduling, the resources are allocated to the process for a limited amount of time and then taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. That process stays in the ready

queue till it gets its next chance to executed.

Algorithms based on preemptive scheduling are: Round Robin (RR), Shortest RemainingTime First (SRTF), Priority (preemptive version), etc.

Non-Preemptive Scheduling: In this scheduling, once the resources (CPU cycles) are allocated to a process, the process holds the CPU till it gets terminated or reaches a waiting state.

Algorithms based on non-preemptive scheduling are: Shortest Job First (SJF basically nonpreemptive) and Priority (non-preemptive version), etc.

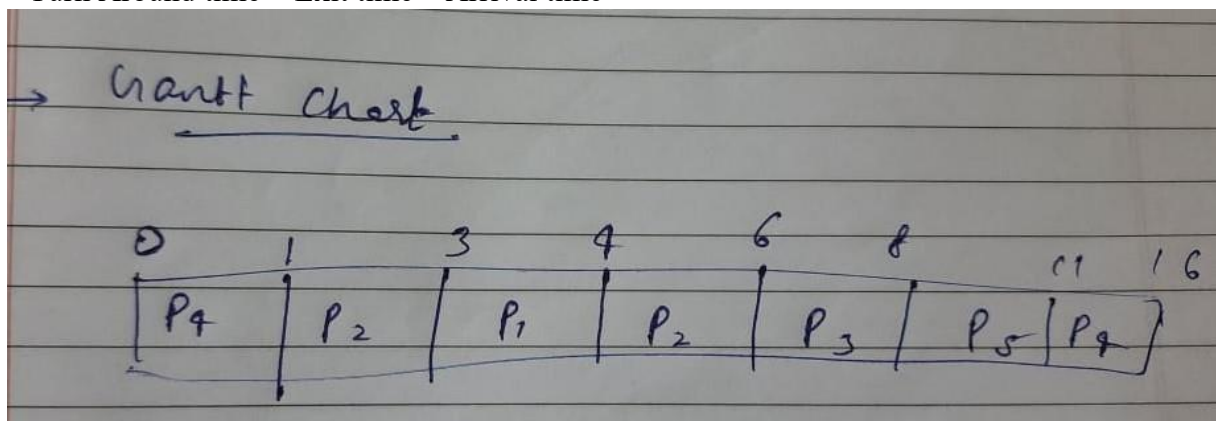
Q17. Consider the collection of these five processes, whose arrival andburst times are provided :-

P Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Calculate the average waiting time and average turnaround time ifthe CPU scheduling policy is SJF preemptive.

Ans.-

Turn Around time = Exit time – Arrival time



Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
------------	-----------	------------------	--------------

P1	4	$4 - 3 = 1$	$1 - 1 = 0$
P2	6	$6 - 1 = 5$	$5 - 4 = 1$
P3	8	$8 - 4 = 4$	$4 - 2 = 2$
P4	16	$16 - 0 = 16$	$16 - 6 = 10$
P5	11	$11 - 2 = 9$	$9 - 3 = 6$

Average Turn Around time = $(1 + 5 + 4 + 16 + 9) / 5 = 35 / 5 = 7$ unit

Average waiting time = $(0 + 1 + 2 + 10 + 6) / 5 = 19 / 5 = 3.8$ unit

Q18. Why I/O jobs are treated as a series of separate jobs?

Q19. Why CPU-bound jobs aren't given priority for response time?

Q20. At what end does new jobs enter MLFQ?

Q21. Is starvation possible in MLFQ?

Q22. What is the maximum time does a process can be without getting scheduled in MLFQ?

Q23.

Consider three process, all arriving at time zero, with total execution time of 10, 20 and 30 units respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of does the CPU remain idle?

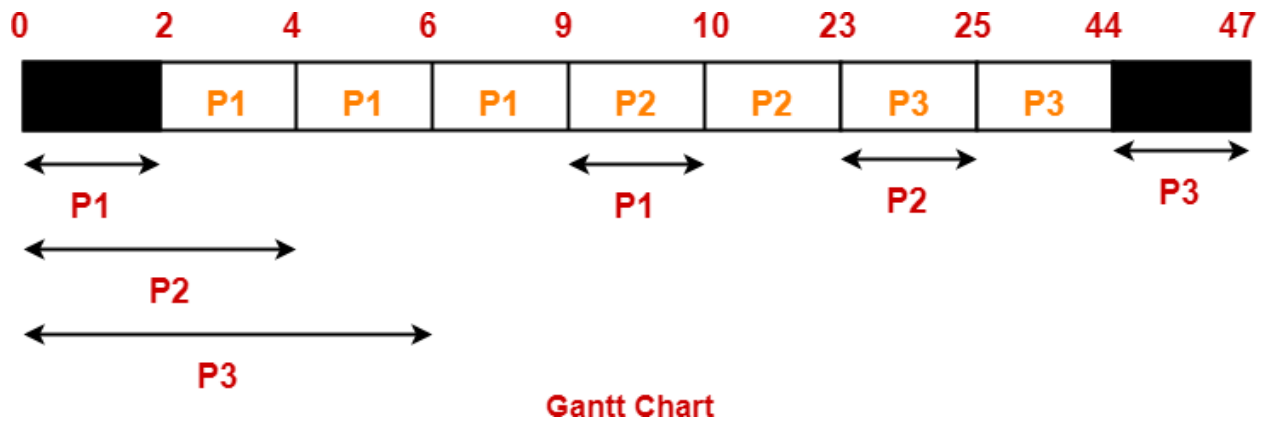
Solution :

According to question, we have-

	Total Burst Time	I/O Burst	CPU Burst	I/O Burst
Process P1	10	2	7	1
Process P2	20	4	14	2
Process P3	30	6	21	3

The scheduling algorithm used is Shortest Remaining Time First.

Gantt Chart-



Percentage of time CPU remains idle
 $= (5 / 47) \times 100$
 $= 10.638\%$

Problem-24

Consider the set of 6 processes whose arrival time and burst time are given below

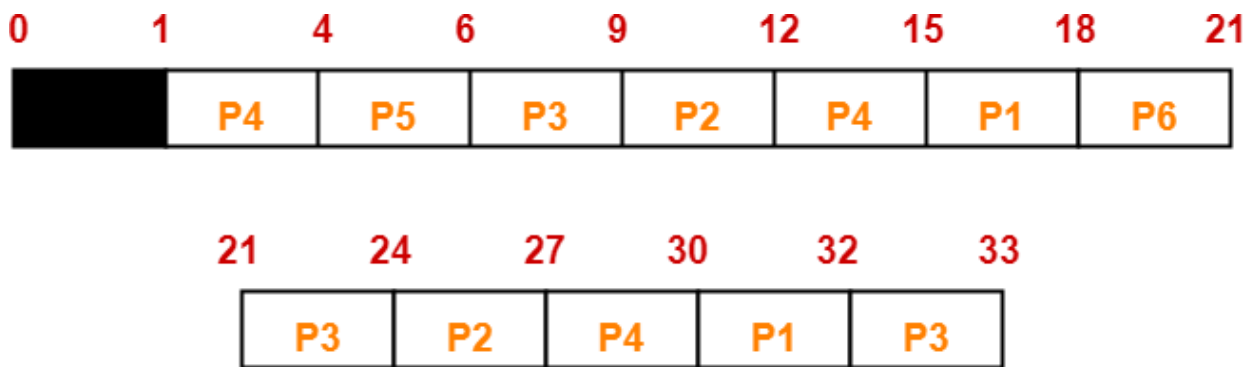
Process Id	Arrival time	Burst time
P1	5	5
P2	4	6
P3	3	7
P4	1	9
P5	2	2
P6	6	3

If the CPU scheduling policy is Round Robin with time quantum = 3, calculate the average waiting time and average turn around time.

Solution :

Ready Queue :

P3, P1, P4, P2, P3, P6, P1, P4, P2, P3, P5, P4



Gantt Chart

Now, we know-

- Turn Around time = Exit time – Arrival time
- Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	32	$32 - 5 = 27$	$27 - 5 = 22$
P2	27	$27 - 4 = 23$	$23 - 6 = 17$
P3	33	$33 - 3 = 30$	$30 - 7 = 23$
P4	30	$30 - 1 = 29$	$29 - 9 = 20$
P5	6	$6 - 2 = 4$	$4 - 2 = 2$
P6	21	$21 - 6 = 15$	$15 - 3 = 12$

Now,

- Average Turn Around time = $(27 + 23 + 30 + 29 + 4 + 15) / 6 = 128 / 6 = 21.33$ unit

- 27) What are the scheduling criteria for different algorithms?
- 28) Explain SJF, FCFS, Priority and Round Robin Algorithm and state the major difference between them.
- 29) In which Algorithm the starvation condition is most likely to occur?
- 30) Describe the functioning of clock, hardware support is needed, What data structures can be used?
- 31) If a system is thrashing, how can we try to reduce thrashing within the OS?
- 32) How does LRU works?
- 33) Can clock behave like a LRU?
- 34) What replacement policy can we use in worst-case LRU situation?
- 35) What is a cooperative approach to scheduling? Is it beneficial?
- 36) Define the term dispatch latency?
- 37) Why there is bad turnaround time in round robin ?
- 38) When will we prefer round robin over STCF ?

39) write a C program for implementation of Priority scheduling algorithms?

I/O

Q1. Why does Write buffering improves performance?

Q2. What is difference between spooling and buffering?

Q3. What is input spooling? Differentiate between Process switch vs context switch?

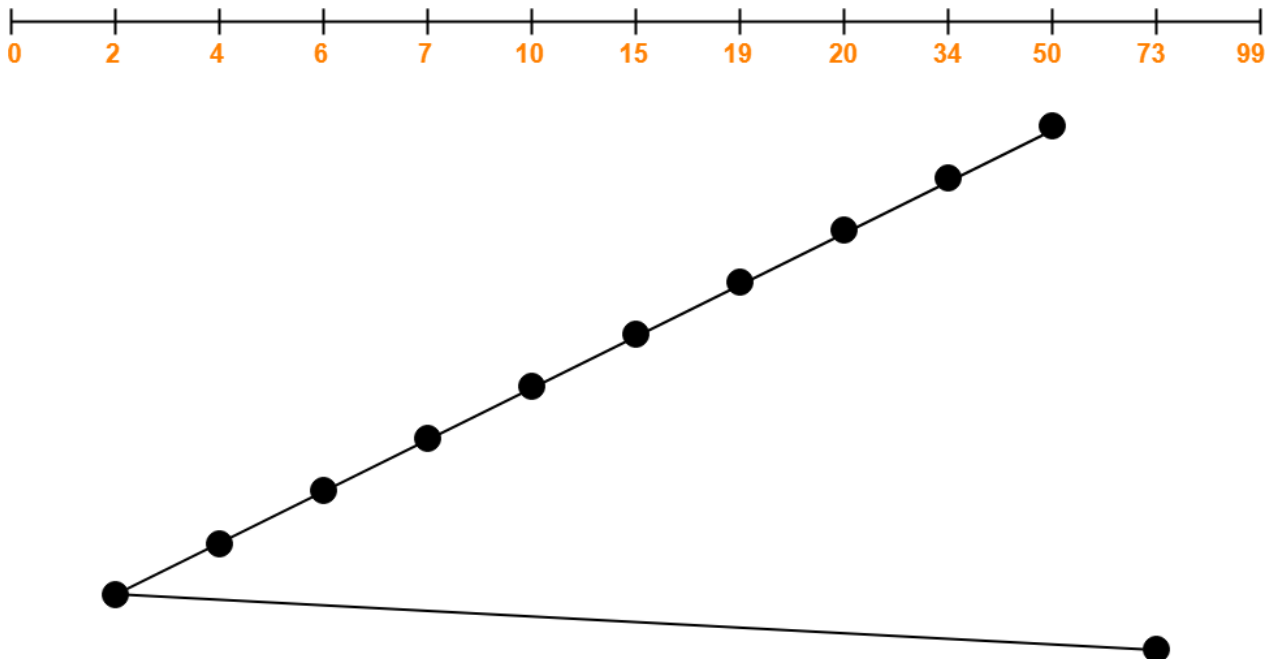
Q4. What is the purpose of an I/O status information?

Q5. Consider a disk system with 100 cylinders. The requests to access the cylinders occur in following sequence-

4, 34, 10, 7, 19, 73, 2, 15, 6, 20

Assuming that the head is currently at cylinder 50, what is the time taken to satisfy all requests if it takes 1 ms to move from one cylinder to adjacent one and shortest seek time first policy is used?

Solution :



Total head movements incurred while servicing these requests

$$\begin{aligned} &= (50 - 34) + (34 - 20) + (20 - 19) + (19 - 15) + (15 - 10) + (10 - 7) + (7 - 6) + (6 - 4) \\ &+ (4 - 2) + (73 - 2) \\ &= 16 + 14 + 1 + 4 + 5 + 3 + 1 + 2 + 2 + 71 \\ &= 119 \end{aligned}$$

Time taken for one head movement = 1 msec. So, Time taken for 119 head movements

$$= 119 \times 1 \text{ msec}$$

= 119 msec

3) Can we say MLFQ CPU scheduling from learns from past behaviour of the process and then predicts its future behaviour? Provide reasons for your answer.

4) What can you say about adjusting T_q to trade off in Round Robin policy scheduling?

2) On considering the Pages reference order by CPU as 0 2 1 0 3 1 0 3 2 1 0 3. Calculate hit % in the case of FIFO and Optimal replacement policy when no of frames is i) 3 ii) 4. Explain what you observed and how can it support Belady's anomaly?

Q26: A certain moving arm disk-storage device has the following specifications: Number of tracks per surface = 4004 Track storage capacity = 130030 bytes Disk speed = 3600 rpm Average seek time = 30 m secs. Estimate the average latency the disk storage capacity and the data transfer rate.

1. Sol: Avg Latency = $12 \times 60 R = 12 \times 60 \times 3600 = 8.33$ ms

2. Disk Storage Capacity = *(We need a number of surface to calculate it)*

$404 \times 130030 \text{ Bytes} \approx 50$ MB per surface (approx)

3. Data transfer rate = Track capacity $\times R = 130030 \times 3600 / 60 = 7801.8$ kBps

Q27: Consider three processes (process id 0, 1, 2 respectively) with compute time bursts 2, 4 and 8 time units. All processes arrive at time zero. Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. The average turn around time is:

Sol: Let the processes be p_0 , p_1 and p_2 . These processes will be executed in following order.

$p_2 \ p_1 \ p_2 \ p_1 \ p_2 \ p_0 \ p_1 \ p_2 \ p_0 \ p_1 \ p_2$

0 4 5 6 7 8 9 10 11 12 13 14

Turn around time of a process is total time between submission of the process and its completion.

Turn around time of $p_0 = 12$ (12-0)

Turn around time of $p_1 = 13$ (13-0)

Turn around time of $p_2 = 14$ (14-0)

Average turn around time is $(12+13+14)/3 = 13$.

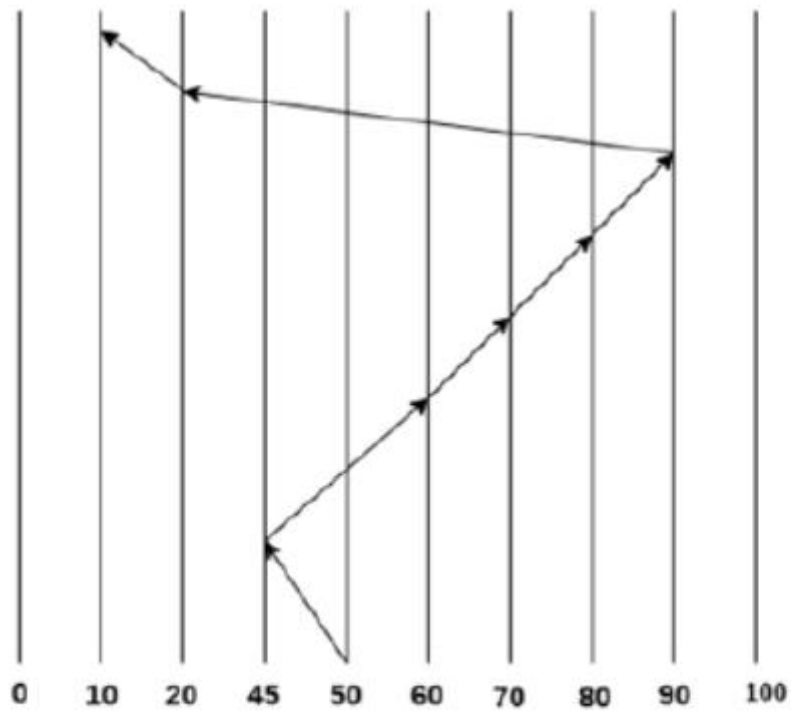
Q28: Suppose the following disk request sequence (track numbers) for a disk with 100 tracks is given: 45, 20, 90, 10, 50, 60, 80 and 70. Assume that the initial position of the R/W head is on track 50. The additional distance that will be traversed by the R/W head when the Shortest Seek Time First (SSTF) algorithm is used compared to the SCAN (Elevator) algorithm (assuming that SCAN algorithm moves towards 100 when it starts execution) is _____ tracks

Sol: using SSTF algorithm

Number of track are 100.

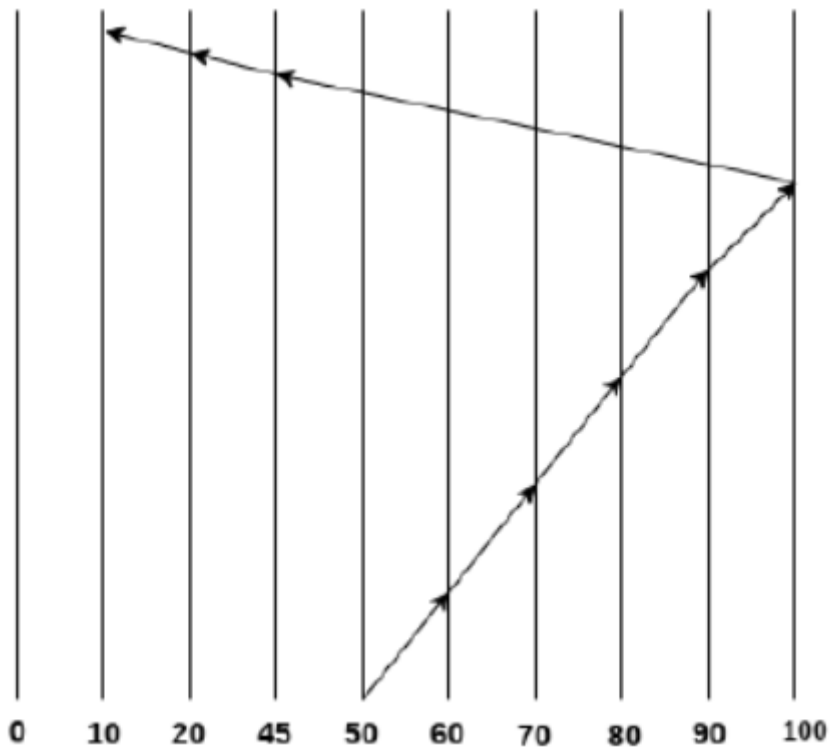
Initial Position of R/W head is 50.

The requests are: 45, 20, 90, 10, 50, 60, 80 and 70



Number of crossed cylinders = $5 + 15 + 10 + 10 + 10 + 70 + 10 = 130$

Using SCAN Algorithm



Number of cylinders crosses = $0 + 10 + 10 + 10 + 10 + 10 + 55 + 25 + 10 = 140$
 The SCAN algorithm travels for 10 additional tracks.

Kernel

Q1. What happens if the Kernel trust on the User stack to restore the state of a process?

Q2. Describe how interrupts are implemented.

Q3. What are the different types of Kernel?

- Monolithic Kernels –In this architecture of kernel, all the system services were packaged into a single system module which lead to poor maintainability and huge size of kernel.
- Microkernels – They follow the modular approach of architecture. Maintainability became easier with this model as only the concerned module is to be altered and loaded for every function. This model also keeps a tab on the ever growing code size of the kernel.

Q4: What is the difference between Kernel and OS?

Kernel: Kernel is a system program that controls all programs running on the computer. The kernel is basically a bridge between the software and hardware of the system.

Operating System: Operating system is a system program that runs on the computer to provide an interface to the computer user so that they can easily operate on the computer.

Kernel	OS
It is considered a central component of OS	It is considered system software.
It is generally responsible for converting user commands into machine-level commands.	It is generally responsible for managing the resources of system.
It simply acts as an interface between hardware and applications.	It simply acts as an interface between hardware and user.
It also performs functions like process management, file management, device management, I/O communication, etc.	It also performs functions like providing security to data and files in the the system, providing access controls to users, maintaining the system privacy, etc.
Its type includes Microkernel, Monolithic kernel, etc.	Its type includes Single and Multiprogramming batch systems, Distributed OS, Real-time OS.

5) Give examples of process handled by Kernel, OS and Users.

6) Write basic code of shell.

7) Which is true about operating systems?

- They always make systems run faster
- They never crash
- They generally make systems easier to use
- None of the above

7) In UNIX, what is used to set the task priority?

8) What is root partition?

9) what is monolithic kernel

10) What is difference between monolithic os and micro-kernel os ?